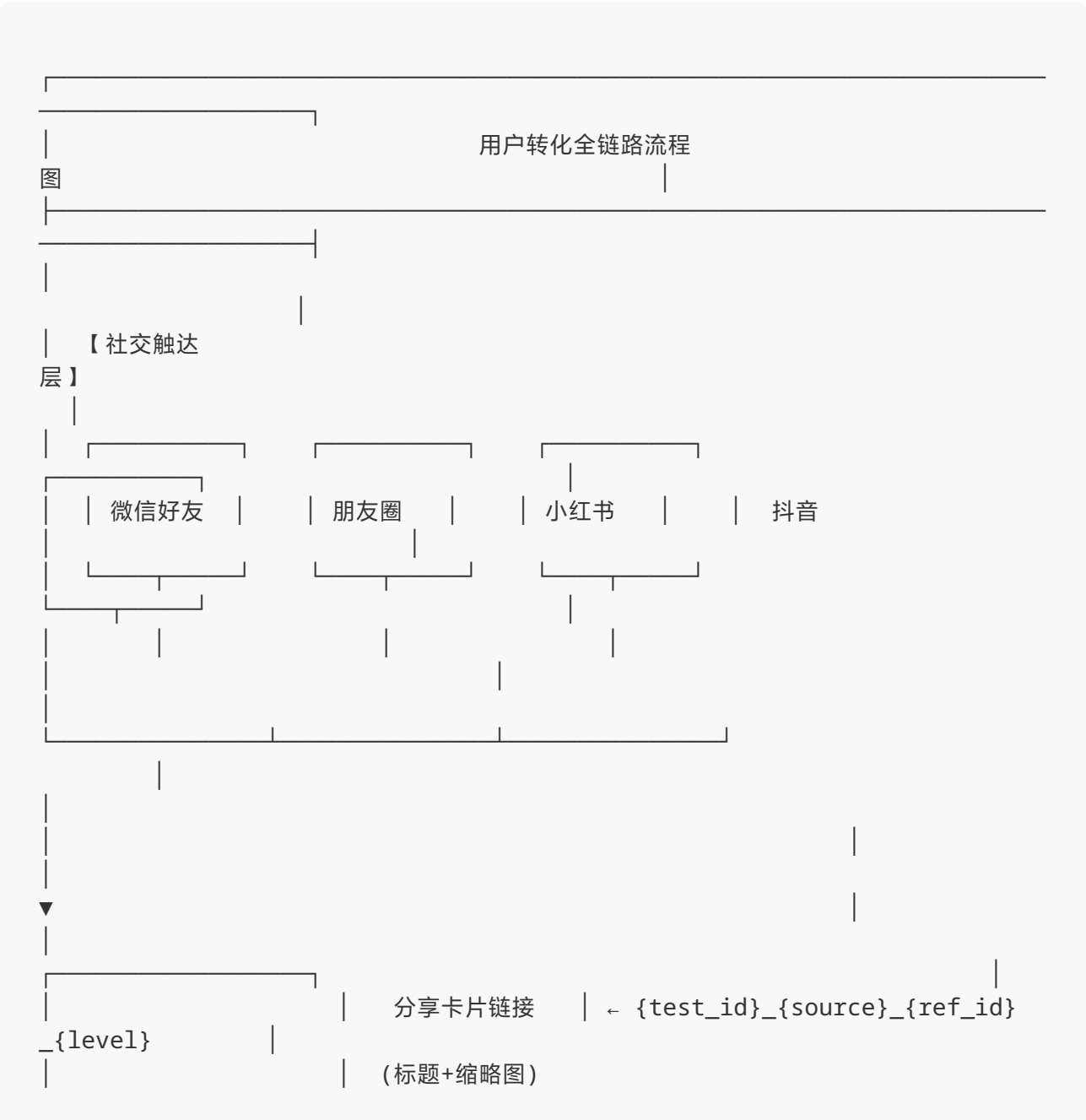


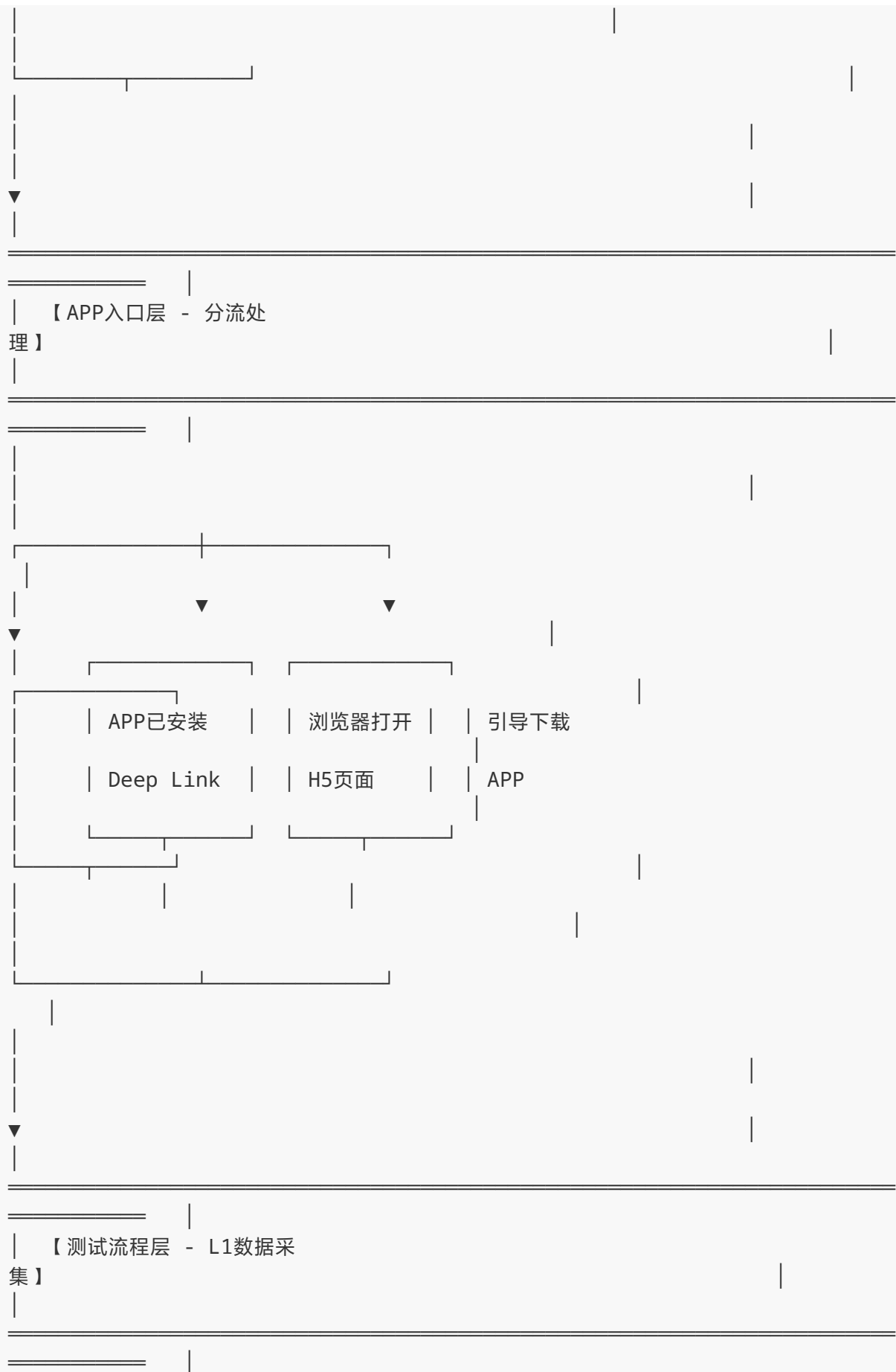
英语诊断测试与转化系统 - 扣子(Coze)移动端APP开发指令

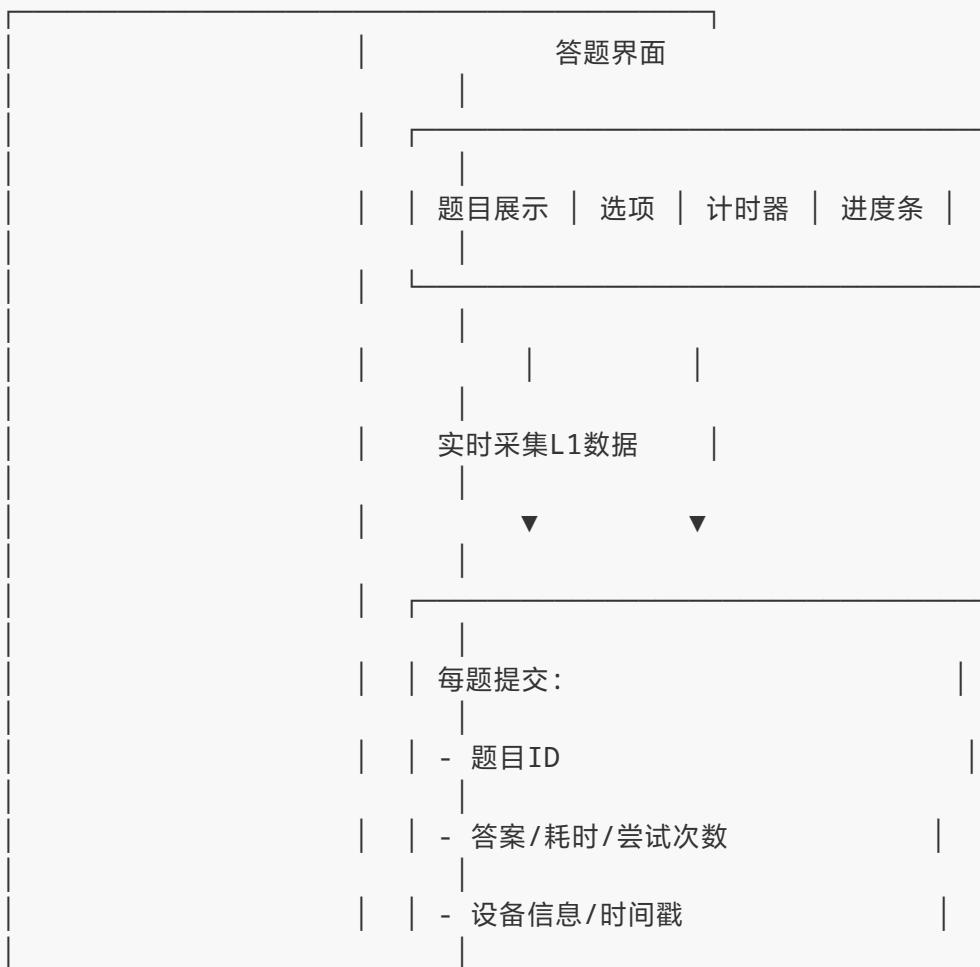
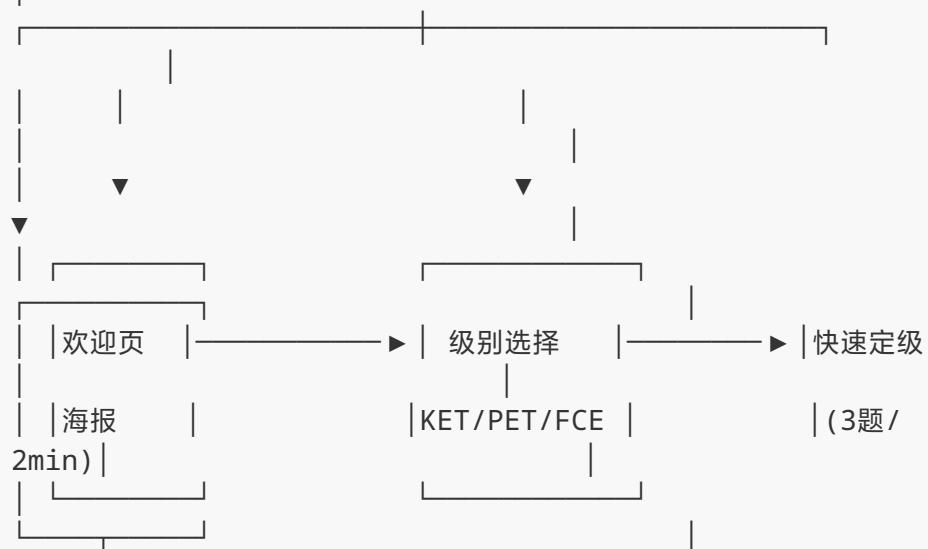
版本: v1.0
更新日期: 2025年1月
文档目标: 为扣子(Coze)平台开发者提供可直接编码的完整开发指令

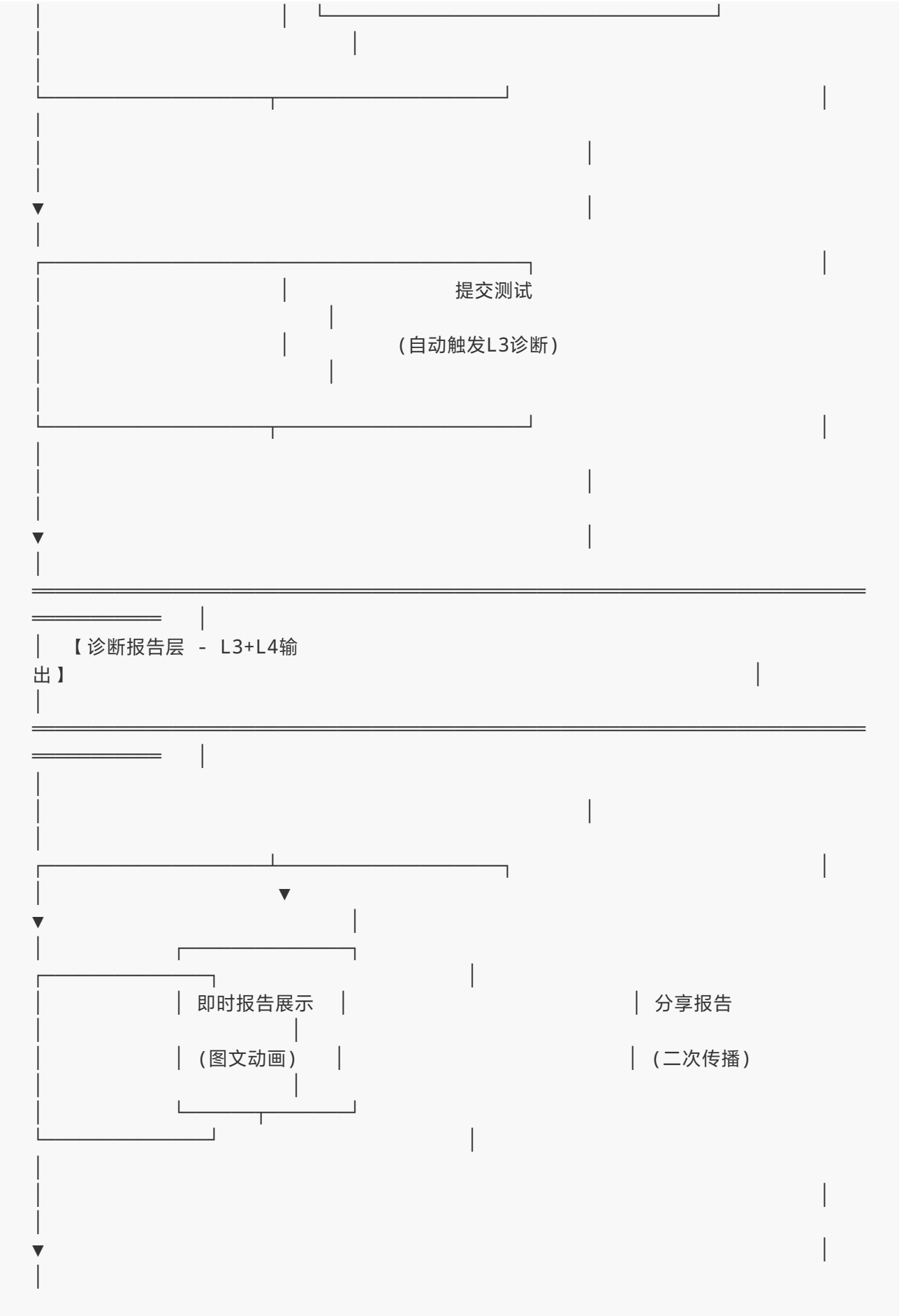
1. 产品功能全景图

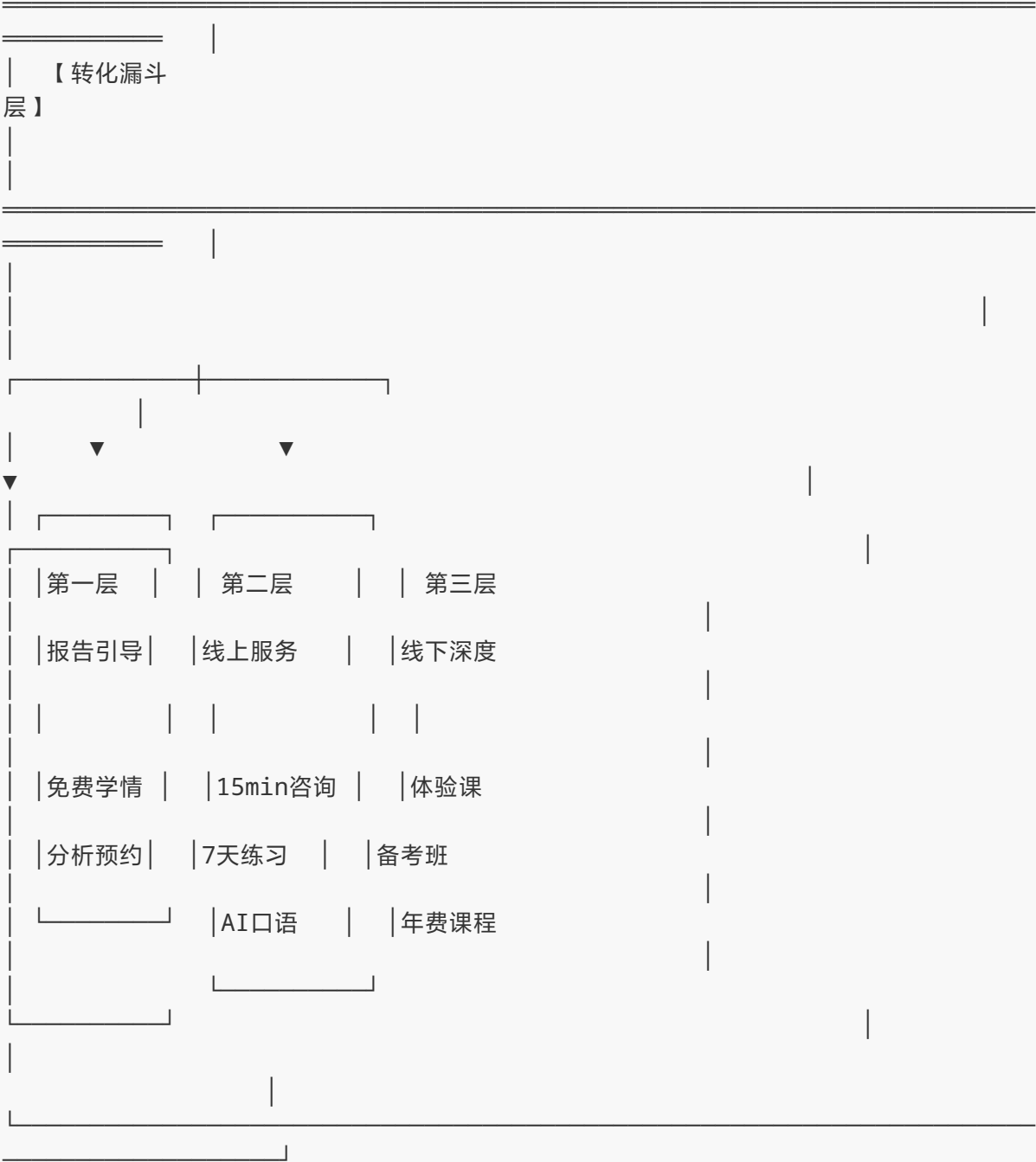
1.1 完整用户路径流程图











1.2 数据流向矩阵

环节	用户行为	系统动作	数据流向	L层
分享点击	扫码/点击链接	解析URL参数，记录来源	test_click_logs	L1
APP打开	唤起APP/下载引导	device_binding + init_session	user_devices	L1

环节	用户行为	系统动作	数据流向	L层
级别选择	选择KET/PET/FCE	记录level_choice	test_sessions	L1
快速定级	完成3题测试	IRT参数估算	diagnostic_results	L3
正式答题	每题作答+提交	实时写入answer_logs	answer_logs	L1
测试提交	点击提交	触发L3诊断+L4画像	diagnostic_results	L3/L4
报告生成	查看报告	渲染报告模板+缓存	ability_profiles	L4
分享报告	分享到社交	记录share_action	conversion_funnel	-
预约咨询	点击预约	创建lead+通知销售	leads	-
线上服务	使用服务	采集使用行为	service_usage	L1

2. 分享测试链接功能

2.1 分享卡片设计规范

2.1.1 微信分享卡片



36名学员验证 · 家长力荐

[小程序图标] 坚果英语诊断

技术规格：

- 标题：最长14个汉字，超出截断
- 描述：最长40个汉字，超出截断
- 缩略图：120x120像素，JPG/PNG，小于128KB
- 跳转页面：<pages/test/entry?scene=xxx>

2.1.2 小红书分享格式

标题：#英语学习 #KET备考 3分钟免费测试！测出孩子真实英语水平

正文：

🎯【免费英语能力诊断】

✅ 基于KET/PET/FCE真题

✅ AI分析薄弱知识点

✅ 生成个性化学习建议

想知道自己孩子英语在什么水平？

点击链接免费测试 ↓

[测试链接]

#少儿英语 #英语启蒙 #剑桥英语

2.1.3 朋友圈分享格式

🎯 免费英语能力诊断 | 3分钟测出水平

基于KET/PET/FCE真题

AI精准分析薄弱点

36名学员验证

[测试链接]

2.2 链接参数设计

2.2.1 URL结构

```
https://{domain}/test/entry?{params}

# 完整示例
https://example.com/test/entry?
test_id=KET_EXP_001&source=wechat&ref_id=user_123&level=KET&channel=fri
end&t=1705234567
```

2.2.2 参数定义

参数名	类型	必填	说明	示例
test_id	string	是	测试模板ID	KET_EXP_001
source	string	是	来源渠道	wechat/ mini_program/ xhs/douyin
ref_id	string	是	推荐人/分享者 ID	user_123 或 ad_campaign_00 1
level	string	否	指定测试级别	KET/PET/FCE (不填则让用户 选择)
channel	string	否	细分渠道	friend/ moments/search
entry_point	string	否	入口点位	banner/chatbot/ article
utm_source	string	否	UTM来源	marketing
utm_medium	string	否	UTM媒介	cpc
utm_campaign	string	否	UTM活动	spring_2025
_t	int64	是	时间戳(秒)	1705234567

参数名	类型	必填	说明	示例
sign	string	是	签名(防篡改)	md5(params+secret)

2.2.3 签名算法

python

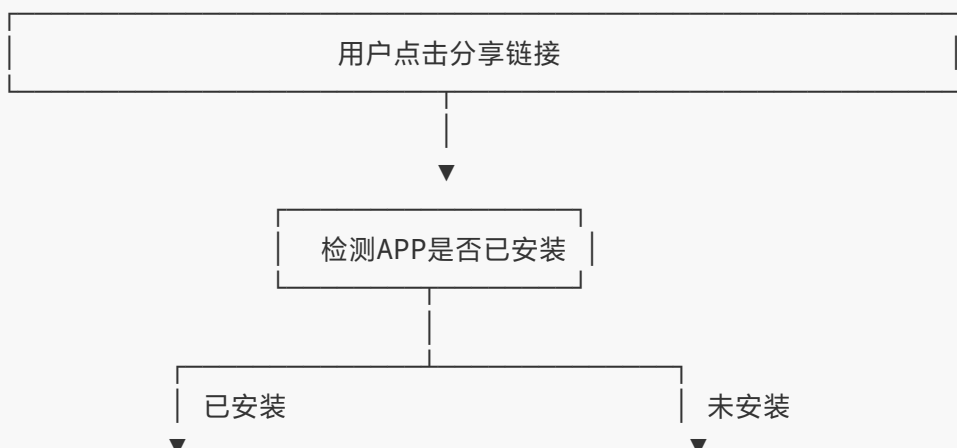
```
import hashlib
import time

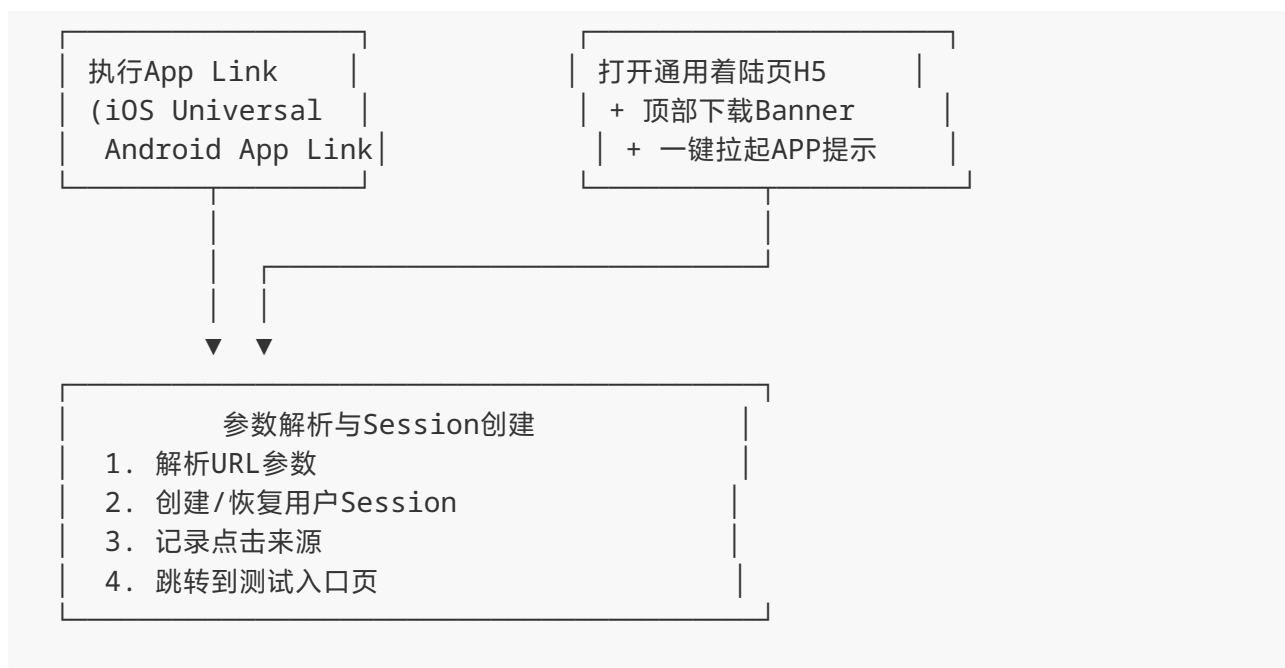
def generate_link_sign(test_id, source, ref_id, secret_key):
    """生成链接签名"""
    raw = f"{test_id}_{source}_{ref_id}_{int(time.time())//86400}"
    return hashlib.md5(f"{raw}_{secret_key}".encode()).hexdigest()[ :16]

def verify_link_sign(params, secret_key):
    """验证链接签名"""
    sign = params.get('sign', "")
    params_copy = params.copy()
    params_copy.pop('sign')
    raw_str = '_'.join([f"{k}={params_copy[k]}" for k in sorted(params_copy)])
    expected = hashlib.md5(f"{raw_str}_{secret_key}".encode()).hexdigest()[ :16]
    return sign == expected
```

2.3 深度链接处理逻辑

2.3.1 路由决策树





2.3.2 各平台实现

iOS Universal Links:

```
json
// apple-app-site-association
{
  "applinks": {
    "details": [{
      "appID": "team.bundle.id",
      "paths": ["/test/entry*"]
    }]
  }
}
```

Android App Links:

```
<!-- AndroidManifest.xml -->
<intent-filter android:autoVerify="true">
  <action android:name="android.intent.action.VIEW" />
  <category android:name="android.intent.category.BROWSABLE" />
  <category android:name="android.intent.category.DEFAULT" />
  <data
    android:scheme="https"
    android:host="example.com"
```

```
xml  android:pathPrefix="/test/entry" />
      </intent-filter>
```

微信内特殊处理:

```
dart
// 微信环境检测与跳转
Future<void> handleWechatDeepLink(Uri uri) async {
  final isWechat = await WxSdk.isWechatInstalled();

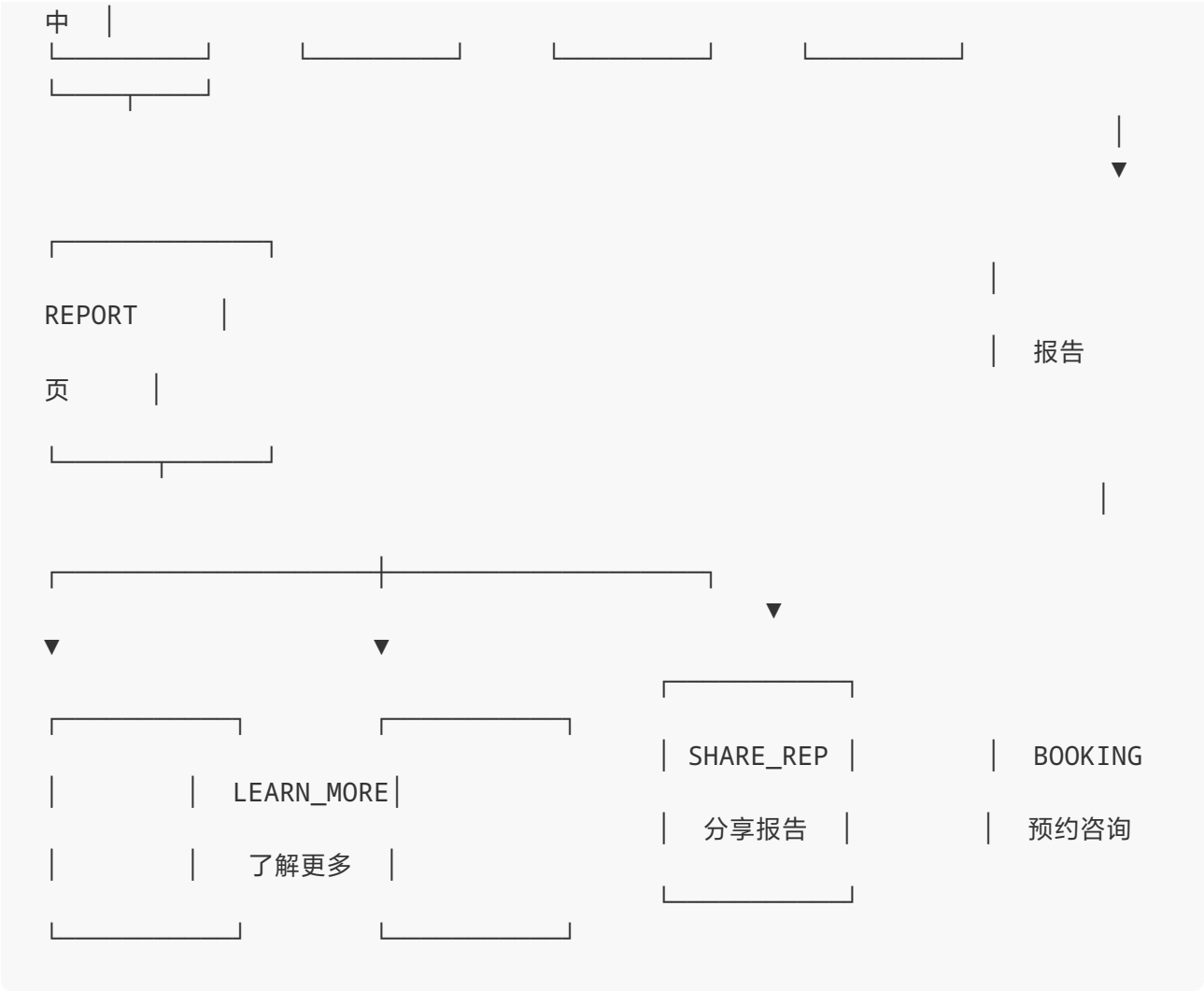
  if (isWechat) {
    // 微信内打开小程序
    if (await canLaunchMiniProgram()) {
      await launchMiniProgram(
        appId: 'wx_xxx',
        path: '/test/entry',
        extraData: uri.queryParameters
      );
    } else {
      // 引导用户长按识别
      showGuideOverlay('长按识别二维码进入测试');
    }
  } else {
    // 浏览器打开
    await launchUrl(uri, mode: LaunchMode.externalApplication);
  }
}
```

3. 测试小程序模块

3.1 测试流程设计

3.1.1 页面流转状态机

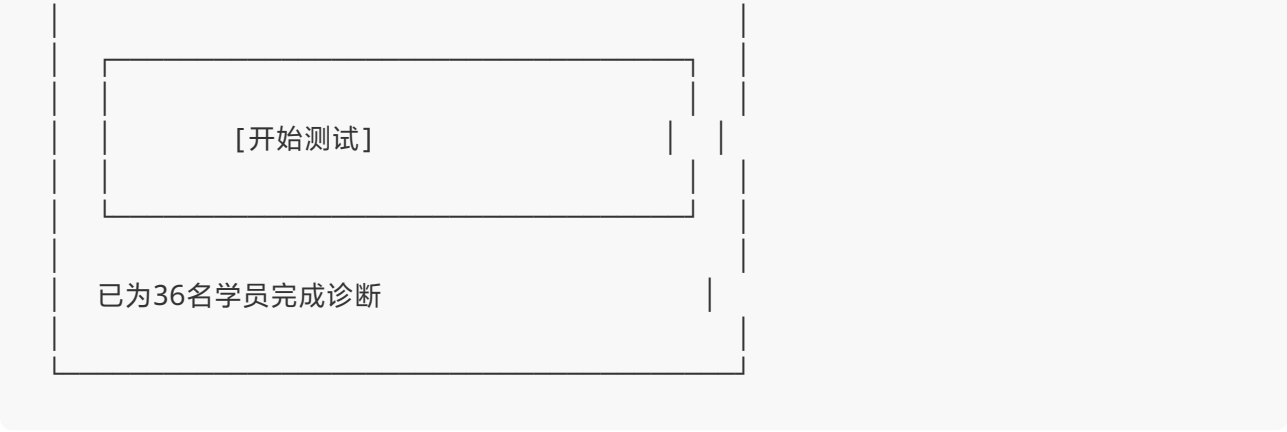




3.1.2 欢迎页 (WELCOME)

页面元素：



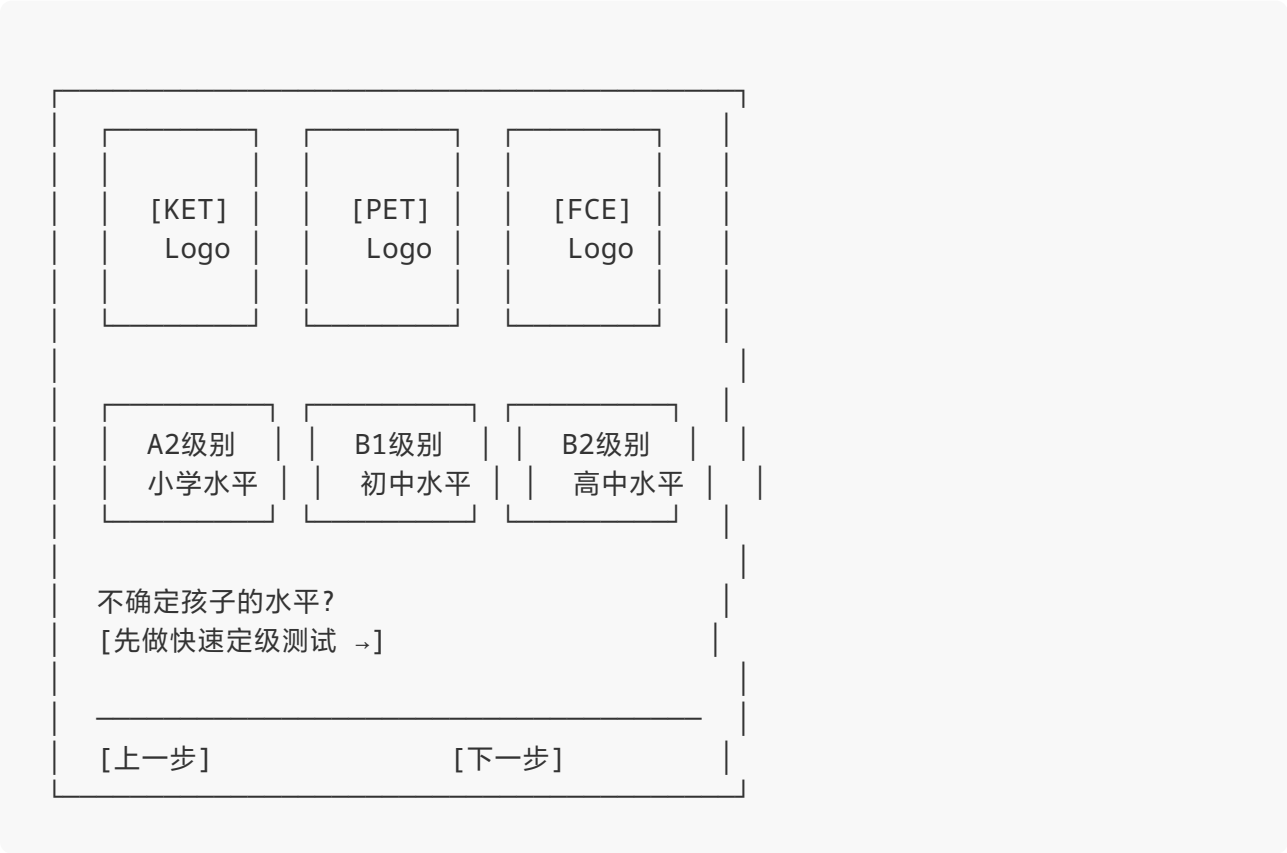


交互逻辑：

- 首次访问：弹出隐私协议+家长知情确认
- 未成年人用户：要求输入家长手机号获取授权码
- 点击"开始测试"：跳转LEVEL_SELECT

3.1.3 级别选择页 (LEVEL_SELECT)

页面元素：



交互逻辑：

- 点击具体级别：记录choice，提交后直接进入对应测试
- 点击"快速定级"：进入PLACEMENT_TEST (3题)
- 自动记忆：localStorage缓存上次选择，下次默认选中

3.2 答题界面规范

3.2.1 主界面布局



3.2.2 交互规范

操作	触发条件	系统响应	动画效果
选择选项	点击选项卡片	高亮选中，发送答案日志	0.2s scale(1.02) + 边框变色
提交答案	点击确认/下一题	记录答案，切换下一题	slideLeft 0.3s
返回上一题	点击"上一题"	加载上一题状态	slideRight 0.3s
修改答案	选择不同选项	覆盖原答案，更新日志	0.15s 颜色闪烁
超时	计时器归零	自动提交当前答案	震动+声音提示
完成答题	最后一题提交	显示loading→跳转报告	淡入+粒子动画

3.2.3 题目类型渲染

选择题：

```
json
{
  "type": "choice",
  "question": "She _____ to school every morning.",
  "options": [
    {"key": "A", "text": "walk"},
    {"key": "B", "text": "walks"},
    {"key": "C", "text": "walking"},
    {"key": "D", "text": "walked"}
  ],
  "media": {
    "type": "none", // audio/image/none
    "url": ""
  }
}
```

听力题：

```
json
{
  "type": "listening",
  "instruction": "听录音，选择正确的答案",
  "audio_url": "https://cdn.example.com/audio/ket_001.mp3",
  "question": "What did the woman ask Tom to do?",
  "options": [
    {"key": "A", "text": "Call her back"},
    {"key": "B", "text": "Send an email"},
    {"key": "C", "text": "Visit her office"},
    {"key": "D", "text": "Buy some books"}
  ],
  "play_count": 1,
  "allow_replay": true
}
```

完形填空：

```
json
{
  "type": "cloze",
  "context": "Yesterday, Tom ___1___ (go) to the park with his friends. They ___2___ (play) football there.",
  "blanks": [
    {"position": 1, "options": ["go", "goes", "went", "going"]},
    {"position": 2, "options": ["play", "plays", "played", "playing"]}
  ]
}
```

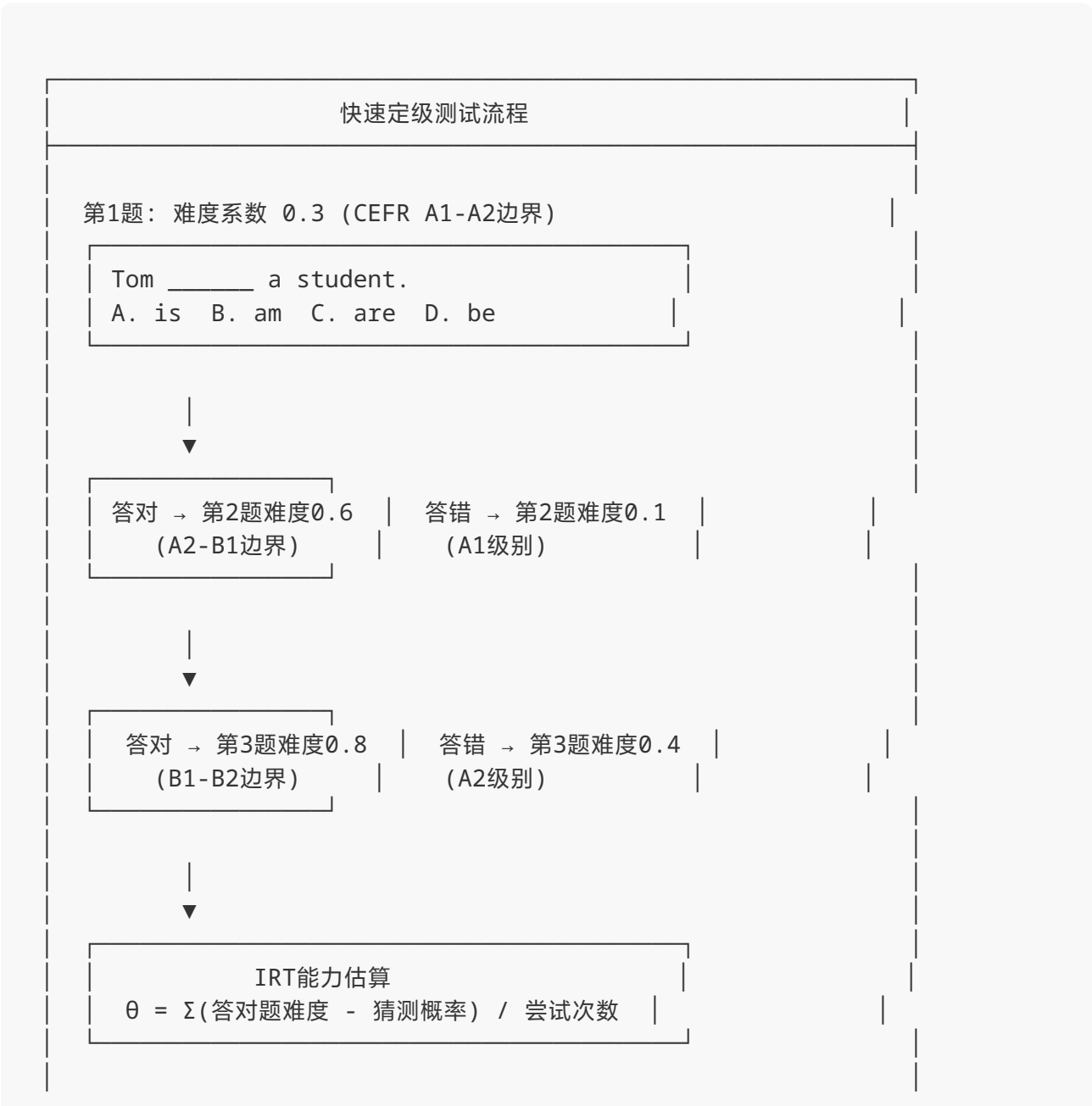
3.3 各级别测试配置

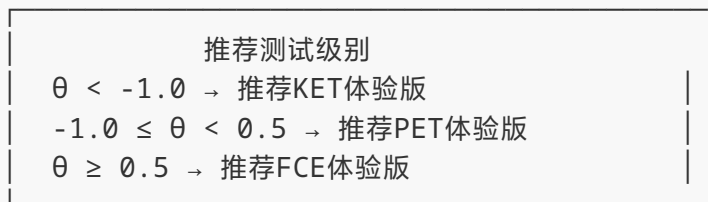
3.3.1 测试规格表

测试类型	题目数量	时间限制	适用场景	题目来源
快速定级	3题	120秒	新用户首次	自适应选题
KET体验版	8题	5分钟	分享引流	精选真题库

测试类型	题目数量	时间限制	适用场景	题目来源
PET体验版	8题	5分钟	分享引流	精选真题库
FCE体验版	8题	5分钟	分享引流	精选真题库
KET完整版	25题	30分钟	深度测评	官方真题
PET完整版	25题	35分钟	深度测评	官方真题
FCE完整版	26题	40分钟	深度测评	官方真题

3.3.2 快速定级测试逻辑





3.3.3 自适应选题算法

```

class AdaptiveQuestionSelector:
    """自适应题目选择器"""

    def __init__(self, question_pool, ability_estimation_model='IRT'):
        self.pool = question_pool  # 题库
        self.model = ability_estimation_model  # IRT或DINA

    def select_next_question(self, current_ability, answered_ids, difficulty_range=(0,1)):
        """
        选择下一道题

        Args:
            current_ability: 当前能力估计值  $\theta$ 
            answered_ids: 已答题目ID列表
            difficulty_range: 难度范围过滤

        Returns:
            question: 选中的题目对象
        """
        # 过滤未答且难度接近当前能力的题目
        candidates = [
            q for q in self.pool
            if q['id'] not in answered_ids
            and difficulty_range[0] <= q['difficulty'] <= difficulty_range[1]
        ]

        if not candidates:
            candidates = [q for q in self.pool if q['id'] not in answered_ids]

        # 按难度差值排序，选择最接近的
  
```

```
python
    for q in sorted(candidates, key=lambda x: abs(x['difficulty'] -
current_ability)):
        # 检查题目曝光间隔 (防刷题)
        if self.check_exposure_gap(q['id']):
            return q

    return candidates[0] if candidates else None

def update_ability(self, question_id, is_correct, prior_ability):
    """
    能力更新

    使用贝叶斯更新或EM算法更新能力估计
    """
    question = self.get_question(question_id)
    difficulty = question['difficulty']
    discrimination = question.get('discrimination', 1.0)

    # IRT  $P(\theta) = c + (1-c) / (1 + \exp(-a(\theta - b)))$ 
    # 答对: 增加能力估计
    # 答错: 降低能力估计
    if is_correct:
        adjustment = (1 - 0.5) * 0.3 * (difficulty - prior_ability)
    else:
        adjustment = -0.3 * (difficulty - prior_ability)

    return prior_ability + adjustment
```

4. 答题数据采集规范

4.1 L1数据采集层接口

4.1.1 数据格式定义

```
{
    "event_type": "answer_submit",
    "session_id": "sess_abc123def456",
    "user_id": "user_12345",           // 匿名用户为null
    "device_info": {
        "device_id": "dev_xxx",
```

```
json    "device_type": "iPhone 15 Pro",
        "os_version": "iOS 17.2",
        "app_version": "1.2.3",
        "screen_size": "393x852"
    },
    "network_info": {
        "network_type": "wifi",
        "ip": "113.x.x.x",
        "region": "广东省-深圳市"
    },
    "test_context": {
        "test_id": "KET_EXP_001",
        "test_type": "experience",           // experience/placement/full
        "level": "KET",
        "source_channel": "wechat",
        "ref_id": "user_123"
    },
    "answer_data": {
        "question_id": "q_001",
        "question_seq": 1,
        "question_type": "choice",
        "difficulty": 0.45,
        "knowledge_point": "grammar_verb_tense",
        "chosen_option": "B",
        "correct_option": "B",
        "is_correct": true,
        "time_spent_ms": 12500,
        "attempt_count": 1,
        "first_select_ms": 3200,             // 首次选择耗时(犹豫度指标)
        "change_count": 0                   // 修改次数
    },
    "timestamp": 1705234567890,
    "client_time": "2025-01-14T15:36:07.890+08:00"
}
```

4.1.2 采集事件类型

事件类型	触发时机	数据特征	采集方式
page_view	页面加载完成	页面ID、来源、停留 时长	实时
test_start	点击开始测试	测试ID、级别、用户 类型	实时

事件类型	触发时机	数据特征	采集方式
question_display	题目展示	题目ID、难度、曝光时间	实时
answer_submit	提交答案	答案、耗时、正确性	实时
test_submit	提交测试	整体耗时、答案完整度	实时
test_abandon	退出测试	退出时机、完成度	实时
report_view	查看报告	查看时长、分享行为	实时
book_consult	预约咨询	预约表单数据	实时

4.1.3 数据采集SDK接入

```
// 数据采集SDK初始化
const tracker = new DiagnosisTracker({
  appId: 'APP_xxx',
  serverUrl: 'https://api.example.com/collect',
  sessionId: getSessionId(),
  enableAutoTrack: true,
  enableSegment: true,
  flushInterval: 3000,
  flushPolicy: 'immediate' // immediate/batch/offline
});

// 页面浏览
tracker.pageView({
  pageId: 'level_select',
  referrer: 'welcome',
  params: { user_type: 'new' }
});

// 答案提交
tracker.track('answer_submit', {
  questionId: 'q_001',
  questionSeq: 1,
  chosenOption: 'B',
  isCorrect: true,
  timeSpentMs: 12500,
  knowledgePoint: 'grammar_verb_tense'
});
```

```

});

// 测试完成
tracker.track('test_complete', {
  testId: 'KET_EXP_001',
  totalTimeMs: 285000,
  correctCount: 6,
  totalQuestions: 8
});

// 报告查看
tracker.track('report_view', {
  reportId: 'rpt_xxx',
  viewDuration: 45000,
  shared: true
});

```

4.2 实时采集 vs 批量上传策略

4.2.1 策略选择



2. 最大缓存1000条
3. 重新联网后按时间顺序上传
4. 超过7天数据自动丢弃

4.2.2 离线缓存实现

```
class OfflineDataCache {
  constructor() {
    this.dbName = 'diagnosis_cache';
    this.storeName = 'events';
    this.maxSize = 1000;
    this.maxAge = 7 * 24 * 60 * 60 * 1000; // 7天
  }

  async init() {
    return new Promise((resolve, reject) => {
      const request = indexedDB.open(this.dbName, 1);
      request.onerror = () => reject(request.error);
      request.onsuccess = () => {
        this.db = request.result;
        resolve();
      };
      request.onupgradeneeded = (e) => {
        const db = e.target.result;
        if (!db.objectStoreNames.contains(this.storeName)) {
          db.createObjectStore(this.storeName, { keyPath: 'id' });
        }
      };
    });
  }

  async save(event) {
    if (!navigator.onLine) {
      // 离线时存入IndexedDB
      await this.addToStore({
        ...event,
        id: `${Date.now()}_${Math.random().toString(36).slice(2)}`,
        cachedAt: Date.now()
      });
    } else {

```

```

javascript      // 在线时直接发送
                await this.sendToServer(event);
            }
        }

        async flush() {
            // 应用启动或联网时调用
            if (!navigator.onLine) return;

            const events = await this.getAllFromStore();
            const validEvents = events.filter(e => Date.now() - e.cachedAt <
this.maxAge);

            for (const event of validEvents) {
                try {
                    await this.sendToServer(event);
                    await this.removeFromStore(event.id);
                } catch (err) {
                    console.error('发送失败:', err);
                    break; // 遇到失败暂停，避免数据乱序
                }
            }
        }
    }
}

```

4.3 数据库表设计

4.3.1 新增表结构

-- 答题行为日志表

```

CREATE TABLE answer_logs (
    id                BIGSERIAL PRIMARY KEY,
    session_id        VARCHAR(64) NOT NULL,
    user_id           VARCHAR(64),
    test_id           VARCHAR(32) NOT NULL,
    question_id       VARCHAR(32) NOT NULL,
    question_seq      SMALLINT NOT NULL,
    question_type     VARCHAR(20) NOT NULL,
    difficulty        DECIMAL(5,3),
    knowledge_point    VARCHAR(64),
    chosen_option      VARCHAR(10),
    correct_option     VARCHAR(10),
    is_correct        BOOLEAN,

```



```

time_spent_ms    INTEGER NOT NULL,
attempt_count   SMALLINT DEFAULT 1,
first_select_ms INTEGER,
change_count    SMALLINT DEFAULT 0,
source_channel  VARCHAR(32),
ref_id          VARCHAR(64),
device_id       VARCHAR(128),
created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_session (session_id),
INDEX idx_user (user_id),
INDEX idx_test (test_id),
INDEX idx_question (question_id),
INDEX idx_knowledge (knowledge_point),
INDEX idx_created (created_at)
);

-- 测试会话表
CREATE TABLE test_sessions (
    id                BIGSERIAL PRIMARY KEY,
    session_id        VARCHAR(64) UNIQUE NOT NULL,
    user_id           VARCHAR(64),
    openid            VARCHAR(128),
    test_id           VARCHAR(32) NOT NULL,
    test_type         VARCHAR(20) NOT NULL, -- experience/placement/full
    level             VARCHAR(10) NOT NULL, -- KET/PET/FCE
    source_channel     VARCHAR(32),
    ref_id            VARCHAR(64),
    device_id         VARCHAR(128),
    device_type       VARCHAR(50),
    os_version        VARCHAR(20),
    app_version       VARCHAR(20),
    ip_address        VARCHAR(45),
    region            VARCHAR(50),
    status            VARCHAR(20) DEFAULT 'in_progress', -- in_progress/
    completed/abandoned
    total_questions   SMALLINT,
    answered_count    SMALLINT DEFAULT 0,
    correct_count     SMALLINT DEFAULT 0,
    total_time_ms     INTEGER,
    placement_result   DECIMAL(5,3), -- 快速定级的能力估计值
    recommended_level VARCHAR(10),
    started_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at      TIMESTAMP,

    INDEX idx_user (user_id),

```

```

INDEX idx_openid (openid),
INDEX idx_status (status),
INDEX idx_source (source_channel),
INDEX idx_created (created_at)
);

-- 诊断报告表
CREATE TABLE diagnostic_reports (
  id BIGSERIAL PRIMARY KEY,
  report_id VARCHAR(64) UNIQUE NOT NULL,
  session_id VARCHAR(64) NOT NULL,
  user_id VARCHAR(64),
  test_id VARCHAR(32) NOT NULL,
  level VARCHAR(10) NOT NULL,

  -- 能力画像 (L4输出)
  ability_theta DECIMAL(5,3) NOT NULL, -- IRT能力参数
  ceفر_level VARCHAR(5) NOT NULL, -- A1/A2/B1/B2/C1/C2
  percentile SMALLINT, -- 百分位排名

  -- 分项能力
  reading_ability DECIMAL(5,3),
  listening_ability DECIMAL(5,3),
  writing_ability DECIMAL(5,3),
  speaking_ability DECIMAL(5,3), -- 需要接入讯飞后完善

  -- 薄弱点定位 (L3输出)
  weak_points JSONB, -- [{"point": "grammar_verb_tense", "level": "weak"}, ...]
  strong_points JSONB,

  -- 报告内容
  report_data JSONB, -- 完整报告数据(含UI渲染信息)
  pdf_url VARCHAR(256),
  share_count SMALLINT DEFAULT 0,

  -- 转化状态
  conversion_status VARCHAR(20) DEFAULT 'report_viewed', -- report_viewed/
consult_booked/service_used/paid
  consult_booking_id BIGINT,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  expires_at TIMESTAMP, -- 报告有效期

  INDEX idx_user (user_id),
  INDEX idx_session (session_id),
  INDEX idx_ceفر (ceفر_level),

```

```

    INDEX idx_conversion (conversion_status),
    INDEX idx_created (created_at)
);

-- 咨询预约表
CREATE TABLE consult_bookings (
    id                BIGSERIAL PRIMARY KEY,
    booking_id        VARCHAR(64) UNIQUE NOT NULL,
    report_id          VARCHAR(64),
    user_id            VARCHAR(64),
    user_name          VARCHAR(50),
    user_phone         VARCHAR(20) NOT NULL,
    child_name         VARCHAR(50),
    child_age          SMALLINT,
    child_grade        VARCHAR(20),
    intention_level    VARCHAR(20) DEFAULT 'medium', -- low/medium/high
    preferred_time      VARCHAR(100),
    source_channel      VARCHAR(32),
    assigned_sales_id   VARCHAR(64),
    booking_status     VARCHAR(20) DEFAULT 'pending', -- pending/confirmed/
    completed/cancelled
    notes              TEXT,
    follow_up_count     SMALLINT DEFAULT 0,
    created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    confirmed_at        TIMESTAMP,
    completed_at        TIMESTAMP,

    INDEX idx_user (user_id),
    INDEX idx_phone (user_phone),
    INDEX idx_status (booking_status),
    INDEX idx_source (source_channel)
);

-- 转化漏斗表
CREATE TABLE conversion_funnel (
    id                BIGSERIAL PRIMARY KEY,
    session_id         VARCHAR(64) NOT NULL,
    user_id            VARCHAR(64),
    report_id          VARCHAR(64),
    booking_id         VARCHAR(64),

    -- 漏斗各阶段
    stage              VARCHAR(30) NOT NULL,
    stage_status        VARCHAR(20) DEFAULT 'entered',
    entered_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    exited_at          TIMESTAMP,

```

```

sql  exit_reason          VARCHAR(50),

    -- 阶段内行为
    action_type           VARCHAR(30),
    action_detail         JSONB,

    INDEX idx_session (session_id),
    INDEX idx_user (user_id),
    INDEX idx_stage (stage),
    INDEX idx_entered (entered_at)
);

```

5. 自动报告生成

5.1 报告生成触发条件

```

class ReportGenerator:
    """自动报告生成器"""

    TRIGGER_CONDITIONS = {
        'test_complete': {
            'min_answered': 3, # 最少完成3题
            'max_time_gap': 300, # 提交间隔不超过5分钟
            'action': 'generate_report'
        },
        'test_abandon': {
            'min_answered': 5, # 中途退出需完成5题以上才生成报告
            'action': 'generate_partial_report'
        }
    }

    def should_generate(self, session_data):
        """判断是否应生成报告"""
        answered = session_data['answered_count']
        test_type = session_data['test_type']
        status = session_data['status']

        if status == 'completed':
            if answered >= self.TRIGGER_CONDITIONS['test_complete']
            ['min_answered']:
                return True, 'full_report'

```

```
python
    if status == 'abandoned':
        if answered >= self.TRIGGER_CONDITIONS['test_abandon']
['min_answered']:
            return True, 'partial_report'

    return False, None
```

5.2 报告内容模板

5.2.1 报告JSON结构

```
{
  "report_id": "rpt_20250114_001",
  "generated_at": "2025-01-14T15:40:00+08:00",
  "valid_until": "2025-01-21T15:40:00+08:00",

  "user_info": {
    "is_anonymous": true,
    "device_id": "dev_xxx",
    "test_level": "KET"
  },

  "ability_summary": {
    "theta": 0.35,
    "cefr": "A2",
    "percentile": 68,
    "comparable_age": "11-12岁",
    "ability_description": "具备扎实的A2水平，能处理日常英语场景"
  },

  "skill_breakdown": [
    {
      "skill": "阅读理解",
      "ability": 0.42,
      "level": "A2",
      "strength": 3,
      "description": "能读懂简单的通知和指示"
    },
    {
      "skill": "听力理解",
      "ability": 0.38,
```

```

        "level": "A2",
        "strength": 2,
        "description": "能理解简单的日常对话"
    },
    {
        "skill": "词汇运用",
        "ability": 0.28,
        "level": "A2",
        "strength": 2,
        "description": "基础词汇量约800-1000词"
    },
    {
        "skill": "语法结构",
        "ability": 0.25,
        "level": "A2",
        "strength": 1,
        "description": "掌握基本时态和句型"
    }
],

"weak_points": [
    {
        "knowledge_point": "一般过去时",
        "topic": "动词时态",
        "correct_rate": 0.33,
        "recommendation": "建议复习规则动词和不规则动词的过去式变化"
    },
    {
        "knowledge_point": "可数/不可数名词",
        "topic": "名词用法",
        "correct_rate": 0.5,
        "recommendation": "注意some/any和a/an的用法区别"
    }
],

"strong_points": [
    {
        "knowledge_point": "一般现在时",
        "topic": "动词时态",
        "correct_rate": 0.83,
        "description": "现在时态运用较为熟练"
    }
],

"comparison": {
    "vs_students": {

```

```
    "percentile": 68,
    "description": "超过了68%的同龄考生"
  },
  "vs_target": {
    "target_level": "PET",
    "gap": 0.85,
    "estimated_time": "6-8个月"
  }
},

"next_steps": [
  {
    "action": "预约学情分析",
    "type": "consult",
    "description": "获取个性化学习规划",
    "priority": 1
  },
  {
    "action": "开始每日一练",
    "type": "practice",
    "description": "巩固基础知识",
    "priority": 2
  }
],

"share_config": {
  "title": "我的英语能力诊断报告",
  "description": "3分钟免费测试，测出真实英语水平",
  "image_url": "https://cdn.example.com/report_share.png",
  "miniprogram_path": "/pages/report/detail?report_id=rpt_xxx"
},

"ui_elements": [
  {
    "type": "radar_chart",
    "data": {
      "labels": ["阅读", "听力", "词汇", "语法"],
      "values": [0.42, 0.38, 0.28, 0.25]
    }
  },
  {
    "type": "progress_bar",
    "data": {
      "current": 68,
      "max": 100,
      "label": "百分位排名"
    }
  }
]
```

```
json
{
  "name": "张三",
  "age": 18,
  "score": 85
}
```

5.2.2 报告页面UI规范

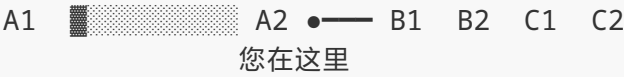
英语能力诊断报告

测试时间：2025年1月14日

【雷达图-能力画像】



CEFR等级



超过 68% 的同龄考生

薄弱知识点

⚠️ 一般过去时 正确率 33% [需加强]
建议：复习不规则动词过去式

⚠️ 可数/不可数名词 正确率 50% [需加强] | |
建议：注意some/any用法

推荐下一步

[免费] 预约1对1学情分析
获得个性化学习规划

[限时] 7天免费口语训练营
AI评测+专业指导

[分享报告给朋友]

[预约咨询]

[再做一次测试]

```

javascript<div class="qrcode-section">
  
  <span class="scan-tip">长按识别查看完整报告</span>
</div>
</div>
`;

// 使用html2canvas或Canvas API渲染
const canvas = await html2canvas(template, {
  width: 750,
  height: 1100,
  scale: 2,
  backgroundColor: '#ffffff'
});

return canvas.toDataURL('image/png');
}

```

5.3.2 二次传播追踪

```

@router.post("/report/share")
async def share_report(report_id: str, share_channel: str) :
    """报告分享接口"""
    # 1. 更新分享计数
    await db.execute(
        """
        UPDATE diagnostic_reports
        SET share_count = share_count + 1
        WHERE report_id = $1
        """,
        report_id
    )

    # 2. 生成带追踪参数的分享链接
    original_report = await db.fetchrow(
        "SELECT * FROM diagnostic_reports WHERE report_id = $1",
        report_id
    )

    share_link = generate_deep_link(
        path="/pages/report/detail",
        params={
            "report_id": report_id,

```

```
python
    "source": f"report_share_{share_channel}",
    "ref_id": original_report['user_id'],
    "_t": int(time.time())
}
)

# 3. 记录分享事件
await db.execute(
    """
INSERT INTO conversion_funnel
(session_id, user_id, report_id, stage, action_type, action_detail)
VALUES ($1, $2, $3, 'report_shared', 'share', $4)
    """,
    session_id, user_id, report_id,
    {"channel": share_channel, "link": share_link}
)

return {"share_link": share_link}
```

6. 转化漏斗设计

6.1 三层转化漏斗架构



成

计：

关键话术设计：

"您的孩子阅读能力已达A2水平，KET备考窗口期已到"

"据诊断，孩子距离PET水平还差约6-8个月系统学习"

"语法基础扎实，但词汇量是当前最大短板"

预约1对1分析

(15min免费)

7天训练营

(低价/免费)

下载学习资料

(资料包)

高意向

中意向

低意向

转化率基准：报告查看→预约咨询 = 5-8%

【第二层】线上轻量服

务

=

的

的

集

值

索

索

点：

点：

点：

点：

点：

点：

点：

点：

习

料

料

服务类型

形式

时长

目

免费学情分析咨询

1v1视频

15min

建立信任+收集需求

每日一练推送

消息推送

7天

持续触达+数据采

AI口语评测体验

语音交互

3次

展示技术+体验价

学习资料包下载

物料

永久

提供价值+留存线

数据采集

- 咨询时：孩子年龄/年级/目标/当前痛点

- 练习时：完成率/正确率/学习时长

- 评测时：发音评分/流利度/完整度

意向度评分规

- 高意向：预约咨询完成 + 完成3次以上练习 + 口语体验完成

- 中意向：预约咨询完成 OR 完成5次以上练

- 低意向：仅查看报告或仅下载资

【第三层】线下深度服务		
意向分层		
转化目标		
触达策略		
高意向(>80分)	体验课	48h内主动电话+优先排课
中意向(50-80)	备考班/课程	推送成功案例+限时优惠
低意向(<50)	持续养客	社群运营+内容触达+节日关怀

6.2 报告内引导策略

6.2.1 动态话术生成

```
class ReportGuidanceEngine:
    """报告内引导话术引擎"""

    def generate_guidance(self, report_data: dict, user_context: dict) -> dict:
        """根据报告数据和用户上下文生成引导策略"""

        cefr = report_data['cefr']
        percentile = report_data['percentile']
        weak_points = report_data['weak_points']
        source_channel = user_context.get('source_channel', 'unknown')

        # 1. 生成CEFR解读
        cefr_insight = self._generate_cefr_insight(cefr, percentile)

        # 2. 生成薄弱点解读
        weak_insights = self._generate_weak_insights(weak_points)
```

python

```
# 3. 生成行动建议
actions = self._generate_actions(cefr, weak_points, user_context)

# 4. 选择最优转化路径
best_action = self._select_best_action(actions)

return {
    "insight": cefr_insight,
    "weak_insights": weak_insights,
    "actions": actions,
    "primary_action": best_action,
    "pop-up_timing": self._get_popup_timing(source_channel),
    "pop-up_style": self._get_popup_style(source_channel)
}

def _generate_cefr_insight(self, cefr: str, percentile: int) -> str:
    """生成CEFR能力解读"""
    insights = {
        "A1": f"处于英语入门阶段，建议从基础词汇和简单句型开始",
        "A2": f"具备日常英语沟通能力，超过了{percentile}%的同龄考生，"
            f"KET备考窗口期已到",
        "B1": f"英语能力达到初中毕业水平，与PET要求还有差距，"
            f"系统学习可加速突破",
        "B2": f"英语能力优秀，FCE水平触手可及，需要专项训练",
        "C1": f"接近母语水平，建议保持训练以冲刺高分"
    }
    return insights.get(cefr, "英语能力评估完成")

def _select_best_action(self, actions: list) -> dict:
    """选择最优转化行动"""
    # 按转化概率和意向匹配度排序
    scored_actions = []
    for action in actions:
        score = action['conversion_probability'] * action['intention_match']
        scored_actions.append((score, action))

    scored_actions.sort(key=lambda x: x[0], reverse=True)
    return scored_actions[0][1] if scored_actions else actions[0]
```

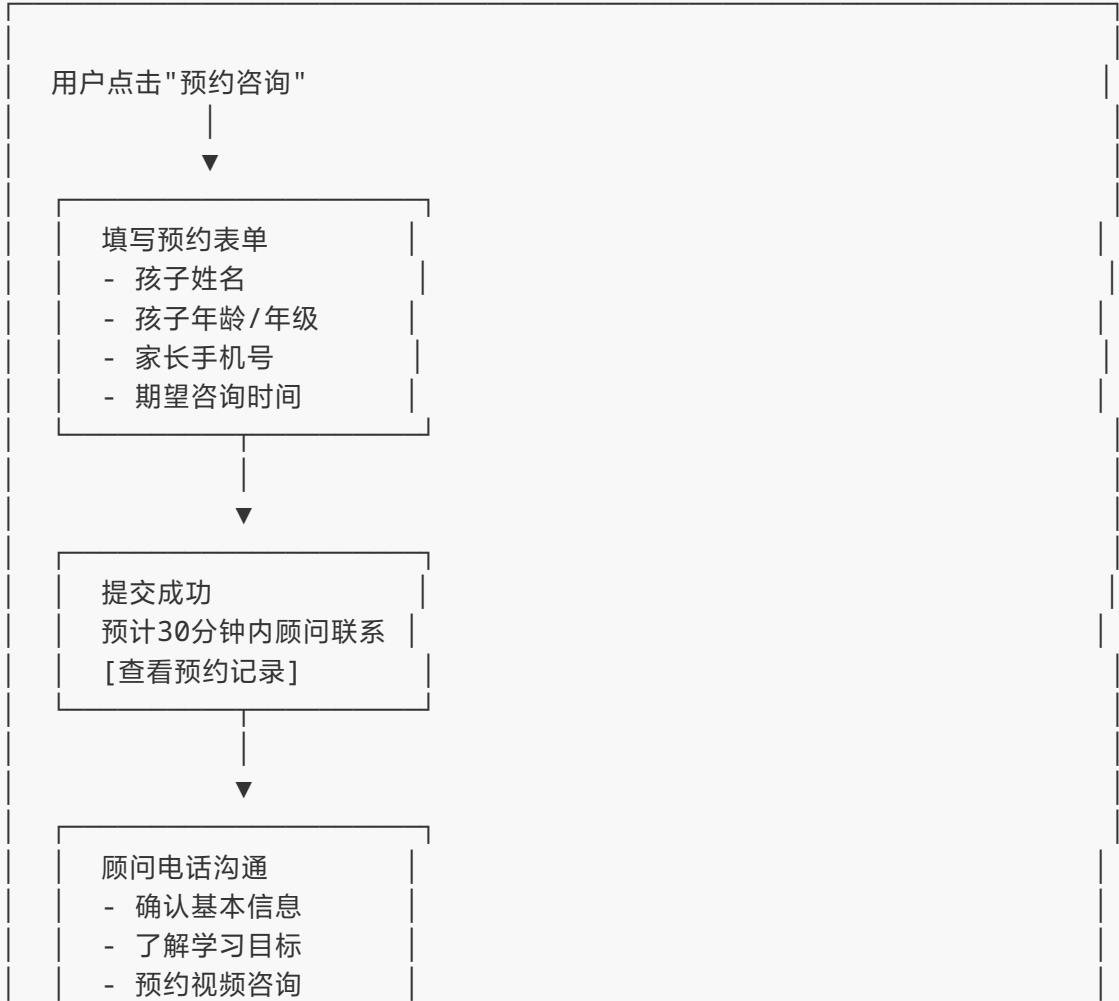
6.2.2 弹窗触发规则

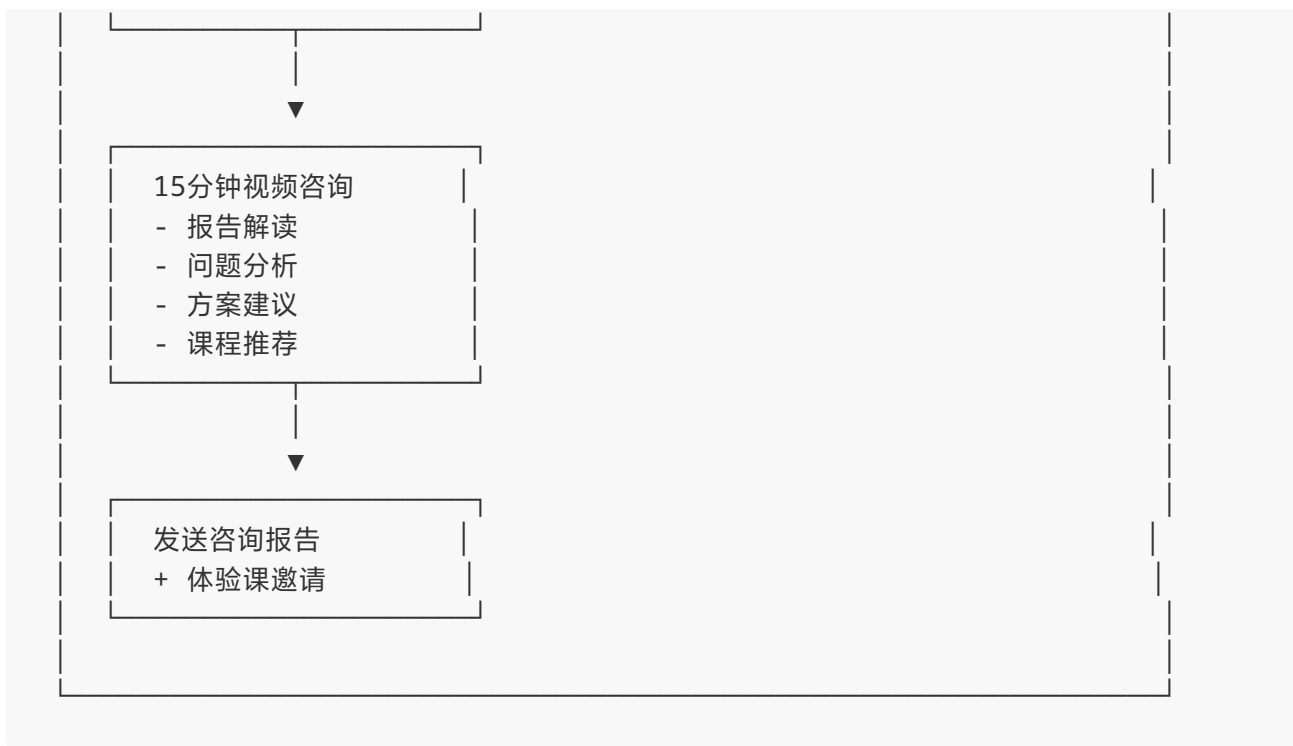
触发条件	触发时机	弹窗内容	关闭后行为
报告查看>20秒	20秒	快速咨询预约	收藏按钮
下滑到"薄弱点"区域	到达时	学习资料下载	隐藏
下滑到"推荐行动"	到达时	预约咨询	底部固定按钮
页面停留>90秒未点击	90秒	咨询弹窗	可关闭
尝试分享报告	点击分享时	分享成功+引导	显示"预约咨询"

6.3 线上服务设计

6.3.1 免费学情分析咨询

服务流程：





6.3.2 每日一练推送

每日一练推送任务

```
DAILY_PRACTICE_TASK = {
  "task_name": "daily_practice_7days",
  "duration": "7天",
  "daily_items": [
    {
      "day": 1,
      "type": "vocabulary",
      "count": 10,
      "content": "基于报告薄弱点生成的词汇练习"
    },
    {
      "day": 2,
      "type": "grammar",
      "count": 5,
      "content": "基于报告薄弱点的语法练习"
    },
    {
      "day": 3,
      "type": "listening",
      "count": 3,
      "content": "对应级别的听力材料"
    }
  ],
}
```

```
python {
    "day": 4,
    "type": "reading",
    "count": 2,
    "content": "阅读理解练习"
},
{
    "day": 5,
    "type": "speaking",
    "count": 3,
    "content": "AI口语评测任务(使用讯飞API)"
},
{
    "day": 6,
    "type": "mock_test",
    "count": 1,
    "content": "模拟测试片段"
},
{
    "day": 7,
    "type": "summary",
    "count": 1,
    "content": "7天学习报告+下一步建议"
}
],
"completion_rewards": {
    "3days": "解锁完整学习资料包",
    "7days": "获得体验课程优惠券"
}
}
```

6.4 筛选与线下引导

6.4.1 意向度评分模型

```
class IntentionScorer:
    """用户意向度评分模型"""

    SCORE_RULES = {
        # 行为积分
        "report_view_complete": 10,    # 完成报告查看
        "share_report": 5,             # 分享报告
        "book_consult": 20,            # 预约咨询
    }
```

```

        "consult_complete": 15,          # 咨询完成
        "practice_start": 5,            # 开始每日练习
        "practice_day3": 10,           # 完成3天练习
        "practice_day7": 15,           # 完成7天练习
        "speaking_trial": 10,          # 完成口语体验
        "download_materials": 3,       # 下载资料

        # 交互深度扣分
        "report_view_short": -5,        # 报告查看<30秒
        "no_interaction_24h": -3,       # 24小时无互动
        "unsubscribe": -20,            # 退订/取消关注

        # 来源渠道调整
        "source_paid_ad": 1.2,          # 付费广告来源乘数
        "source_organic": 1.0,          # 自然流量
        "source_referral": 1.1         # 推荐流量
    }

    def calculate_score(self, user_id: str) -> dict:
        """计算用户意向度评分"""
        # 1. 获取用户全量行为
        behaviors = self.get_user_behaviors(user_id)

        # 2. 累加行为积分
        total_score = 0
        score_breakdown = {}
        for behavior in behaviors:
            points = self.SCORE_RULES.get(behavior['type'], 0)
            total_score += points
            score_breakdown[behavior['type']] = points

        # 3. 乘以渠道系数
        source_channel = self.get_source_channel(user_id)
        channel_multiplier = self.SCORE_RULES.get(f"source_{source_channel}",

1.0)

        final_score = int(total_score * channel_multiplier)

        # 4. 分层
        intention_level = self._classify_intention(final_score)

        return {
            "user_id": user_id,
            "total_score": final_score,
            "intention_level": intention_level, # high/medium/low
            "breakdown": score_breakdown,
            "recommended_action":

```

```

python self._get_recommended_action(intention_level)
    }

    def _classify_intention (self, score: int) -> str:
        """意向度分层"""
        if score >= 80:
            return "high"
        elif score >= 50:
            return "medium"
        else:
            return "low"

    def _get_recommended_action (self, level: str) -> dict:
        """获取推荐行动"""
        actions = {
            "high": {
                "action": "invite_experience_class",
                "urgency": "high",
                "channel": "phone_call",
                "sla": "48小时内必须联系"
            },
            "medium": {
                "action": "send_success_case",
                "urgency": "medium",
                "channel": "wechat_message",
                "sla": "3天内联系"
            },
            "low": {
                "action": "add_to_nurture_group",
                "urgency": "low",
                "channel": "content_marketing",
                "sla": "每周触达"
            }
        }
        return actions.get(level)

```

7. 自动分发测试策略

7.1 分发场景设计

7.1.1 分发场景矩阵

场景	目标用户	测试类型	推送时机	转化目标
新用户首次	未注册/匿名	快速定级	首次访问	收集信息
分享引流	点击分享的用户	体验版测试	立即	完成测试
学员定期	现有36名学员	进阶测试	每月1次	能力跟踪
学员升级	测试达标学员	下一级别体验	测试完成24h	升级转化
沉默用户激活	30天未登录	简短测试	触发沉默	重新激活
活动获客	营销活动参与者	体验版测试	活动期间	完成测试

7.1.2 推送时间优化

```
class PushTimingOptimizer:
    """推送时间优化器"""

    # 用户活跃时段数据(基于历史分析)
    ACTIVE_WINDOWS = {
        "parent_primary": { # 主要决策人(妈妈)
            "weekday": ["12:00-13:00", "20:30-21:30", "22:00-22:30"],
            "weekend": ["09:00-10:00", "14:00-15:00", "19:00-21:00"]
        },
        "parent_secondary": { # 次要决策人(爸爸)
            "weekday": ["07:30-08:00", "21:00-22:00"],
            "weekend": ["10:00-11:00", "15:00-16:00"]
        }
    }

    def get_optimal_time(self, user_profile: dict) -> datetime:
        """获取最优推送时间"""
        is_weekend = datetime.now().weekday() >= 5
        parent_type = user_profile.get('parent_type', 'primary')

        time_windows = self.ACTIVE_WINDOWS[parent_type][
            "weekend" if is_weekend else "weekday"
        ]

        # 随机选择活跃时段
        import random
        selected_window = random.choice(time_windows)
        start, end = selected_window.split('-')
```

```
python
# 在时间窗口内随机选择具体时间
hour = random.randint(int(start.split(':')[0]), int(end.split(':')[0]))

minute = random.choice([0, 15, 30, 45])

tomorrow = datetime.now() + timedelta(days=1)
return tomorrow.replace(hour=hour, minute=minute, second=0)
```

7.2 A/B测试配置

```
AB_TEST_CONFIG = {
    "test_name": "experience_test_version_comparison",
    "variants": {
        "A": {
            "name": "8题精简版",
            "config": {
                "question_count": 8,
                "time_limit": 300,
                "show_timer": True,
                "show_progress": True,
                "allow_feedback": False
            }
        },
        "B": {
            "name": "12题丰富版",
            "config": {
                "question_count": 12,
                "time_limit": 420,
                "show_timer": False,
                "show_progress": True,
                "allow_feedback": True,
                "show_hint": True
            }
        }
    },
    "allocation": {
        "A": 0.5,
        "B": 0.5
    },
    "metrics": {
        "primary": "test_completion_rate",
        "secondary": ["report_view_rate", "consult_booking_rate", "paid_conversion_rate"],
    }
}
```

```
python    "guardrails": ["abandonment_rate", "fake_completion_rate"]
        },
        "sample_size": {
            "per_variant": 500,
            "duration_days": 14
        }
    }
}
```

7.3 学员定期测试分发

```
class StudentTestDispatcher:
    """学员测试分发器"""

    DISPATCH_RULES = {
        "monthly_assessment": {
            "trigger": "schedule",
            "interval": "monthly",
            "test_type": "progression",
            "target_users": "all_active_students",
            "content": "基于当前级别的高阶测试"
        },
        "weakness_reinforcement": {
            "trigger": "auto",
            "condition": "report_weakness_exists",
            "test_type": "targeted_practice",
            "target_users": "specific_students",
            "content": "针对薄弱点的专项测试"
        },
        "level_up_promotion": {
            "trigger": "test_pass",
            "condition": "progression_test_score > 85%",
            "test_type": "next_level_preview",
            "target_users": "qualified_students",
            "content": "下一级别体验测试"
        }
    }

    async def dispatch(self, rule_name: str, context: dict = None):
        """执行分发"""
        rule = self.DISPATCH_RULES.get(rule_name)
        if not rule:
            return
```

```
python
# 1. 确定目标用户
target_users = await self.get_target_users(
    rule['target_users'],
    context
)

# 2. 生成测试链接
test_links = []
for user in target_users:
    link = await self.generate_personalized_link(
        user_id=user['id'],
        test_type=rule['test_type'],
        content=rule.get('content')
    )
    test_links.append({
        "user": user,
        "link": link,
        "scheduled_at": self.calculate_scheduled_time(user)
    })

# 3. 批量发送推送
for item in test_links:
    await self.send_push(item['user'], item['link'], item['scheduled_at'])

# 4. 记录分发日志
await self.log_dispatch_event(rule_name, test_links)
```

8. 扣子编程指令

这是最核心的部分——完整的扣子Coze开发指令，开发者可直接使用此指令进行编码。

8.1 项目概述

项目名称：英语诊断测试与转化系统 (English Diagnosis & Conversion System)
 项目代号：EN-DIAG-APP
 项目类型：移动端应用 (React Native / Flutter)
 目标平台：iOS 13+ / Android 8.0+
 设计风格：简约现代、亲子教育风格

核心功能：

1. 英语能力诊断测试 (KET/PET/FCE)

- 2. AI自适应题目推送
- 3. 即时诊断报告生成
- 4. 用户转化漏斗
- 5. 学员能力画像管理

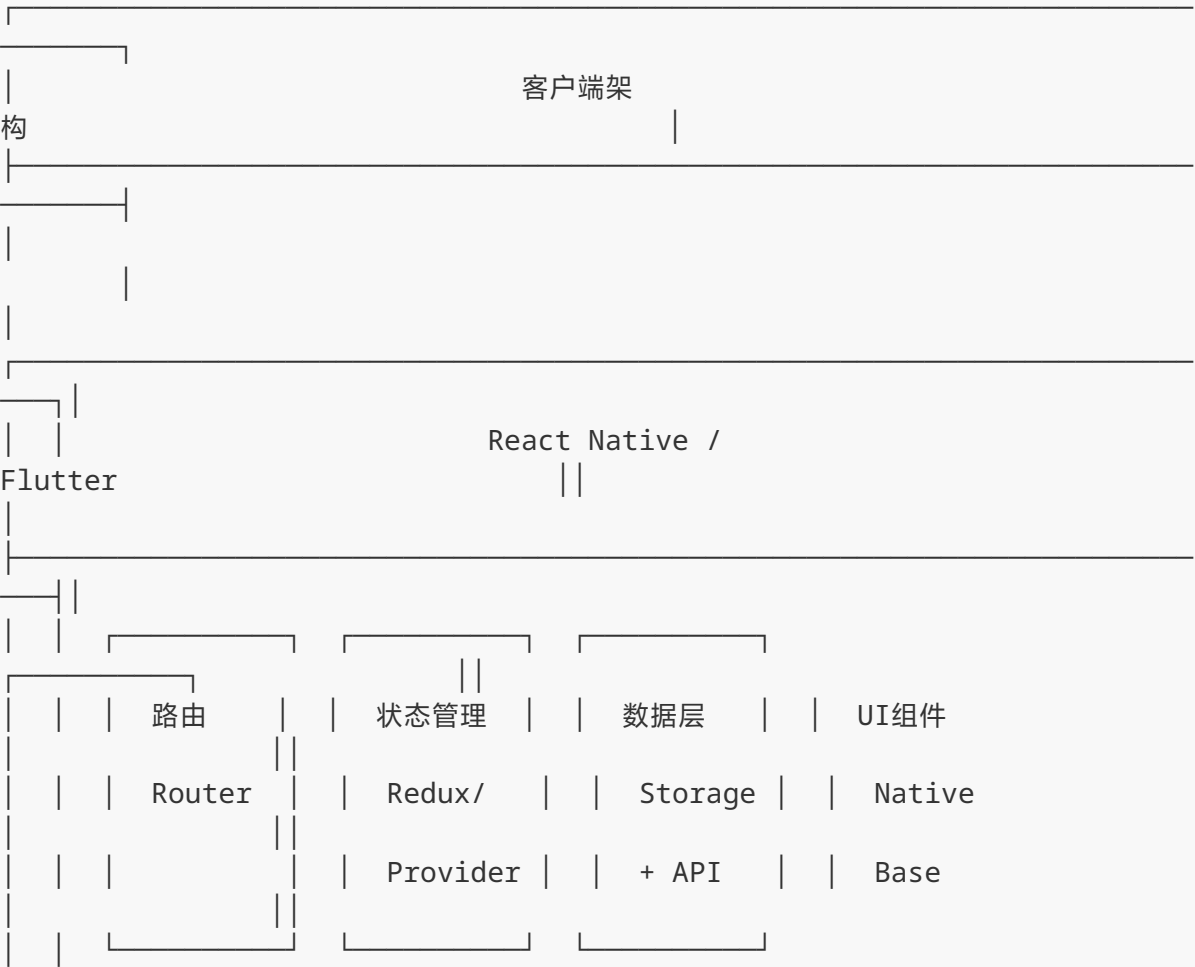
数据架构:

- L1: 数据采集层 (答题行为日志)
- L2: 题库系统 (PostgreSQL)
- L3: 认知诊断模型 (IRT+DINA)
- L4: 学员能力画像 (CEFR)
- L5: 学习规划引擎

外部集成:

- 讯飞语音评测API (口语评测)
- 微信SDK (分享/登录)
- 极光推送 (消息推送)

8.2 技术架构



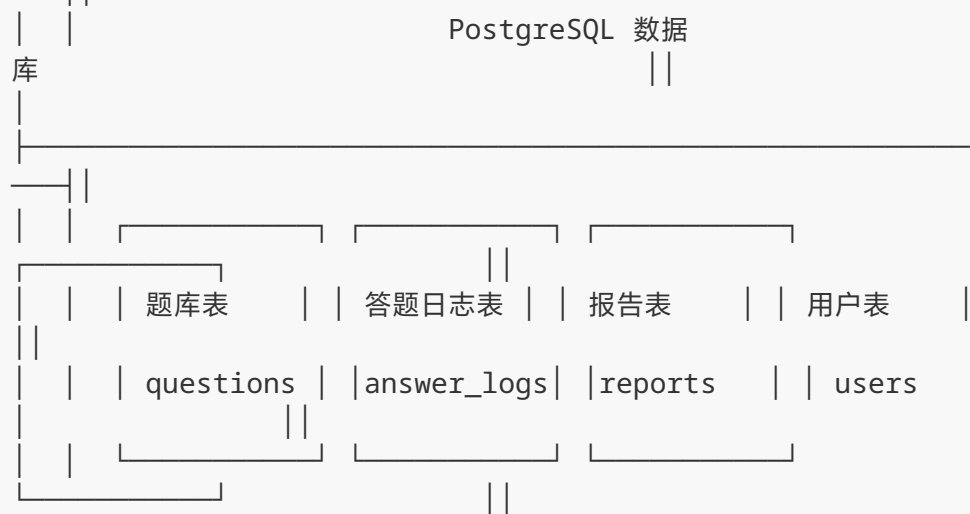
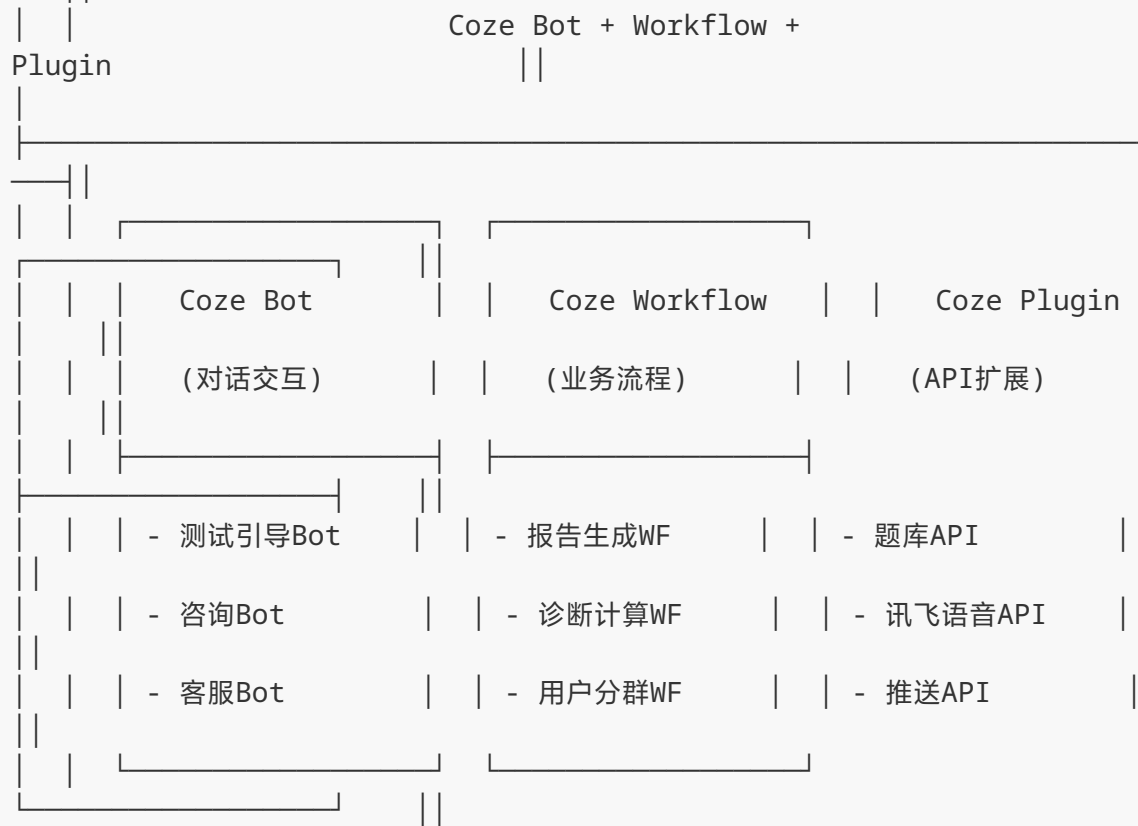
块

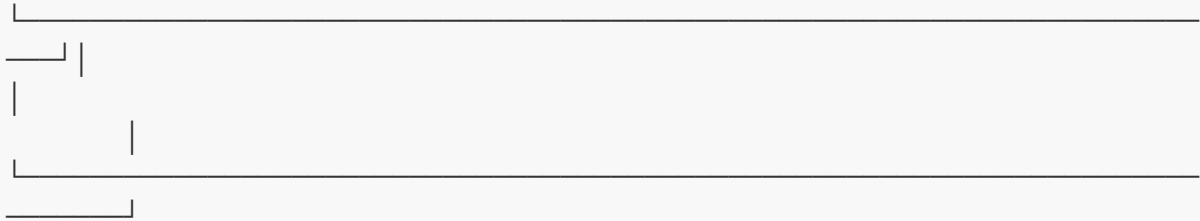
业务模



构

服务端架





8.3 功能模块详细设计

8.3.1 测试模块 (TestModule)

```
// ===== 文件: src/modules/test/TestModule.ts =====

import { EventEmitter } from 'events';

// 题型枚举
export enum QuestionType {
    CHOICE = 'choice',           // 单选题
    MULTIPLE_CHOICE = 'multi_choice', // 多选题
    LISTENING = 'listening',     // 听力题
    CLOZE = 'cloze',             // 完形填空
    READING = 'reading'         // 阅读理解
}

// 测试状态枚举
export enum TestStatus {
    NOT_STARTED = 'not_started',
    IN_PROGRESS = 'in_progress',
    PAUSED = 'paused',
    COMPLETED = 'completed',
    ABANDONED = 'abandoned'
}

// 题目接口
export interface Question {
    id: string;
    type: QuestionType;
    content: string;
    options?: { key: string; text: string; imageUrl?: string }[];
    correctAnswer: string | string[];
    difficulty: number;           // 0-1, IRT难度参数
    discrimination: number;      // 区分度参数
    knowledgePoints: string[]; // 知识点标签
}
```

```

    mediaUrl?: string;           // 听力/图片资源
    explanation?: string;        // 解析
    timeLimit?: number;          // 单题时间限制(ms)
}

// 答题记录
export interface AnswerRecord {
    questionId: string;
    chosenAnswer: string | string[];
    isCorrect: boolean;
    timeSpentMs: number;
    attemptCount: number;
    firstSelectMs: number;        // 首次选择时间(犹豫度)
    changeCount: number;          // 修改次数
    timestamp: number;
}

// 测试会话
export interface TestSession {
    sessionId: string;
    testId: string;
    testType: 'experience' | 'placement' | 'full';
    level: 'KET' | 'PET' | 'FCE';
    status: TestStatus;
    questions: Question[];
    currentIndex: number;
    answers: Map<string, AnswerRecord>;
    startTime: number;
    totalTimeMs: number;
    timeLimit: number;
    sourceChannel?: string;
    refId?: string;
}

// 测试模块主类
export class TestModule {
    private session: TestSession | null = null;
    private eventEmitter: EventEmitter;
    private tracker: DataTracker;
    private questionSelector: AdaptiveQuestionSelector;

    // 公开API
    public on(event: string, handler: Function): void;
    public startTest(config: TestConfig): Promise<void>;
    public submitAnswer(questionId: string, answer: string): Promise<void>;
    public nextQuestion(): Promise<void>;

```

```

typescript
public prevQuestion(): void;
public pauseTest(): void;
public resumeTest(): void;
public abandonTest(): Promise<void>;
public getProgress(): TestProgress;
public getCurrentQuestion(): Question | null;
}

```

// 测试配置

```

export interface TestConfig {
  testId: string;
  testType: 'experience' | 'placement' | 'full';
  level: 'KET' | 'PET' | 'FCE';
  questionCount: number;
  timeLimitSeconds: number;
  sourceChannel?: string;
  refId?: string;
  userId?: string;
  sessionId?: string;
}

```

// 测试进度

```

export interface TestProgress {
  current: number;
  total: number;
  answered: number;
  correct: number;
  timeRemainingMs: number;
  percentage: number;
}

```

8.3.2 报告模块 (ReportModule)

```

// ===== 文件: src/modules/report/ReportModule.ts
=====

```

// CEFR等级

```

export enum CEFRLevel {
  A1 = 'A1',
  A2 = 'A2',
  B1 = 'B1',
  B2 = 'B2',
  C1 = 'C1',
}

```

```

    C2 = 'C2'
}

// 能力维度
export enum AbilityDimension {
    READING = 'reading',
    LISTENING = 'listening',
    WRITING = 'writing',
    SPEAKING = 'speaking',
    VOCABULARY = 'vocabulary',
    GRAMMAR = 'grammar'
}

// 诊断报告
export interface DiagnosticReport {
    reportId: string;
    sessionId: string;
    userId?: string;

    // 能力总览
    abilityTheta: number;           // IRT能力参数
    cefrLevel: CEFRLLevel;
    percentile: number;           // 百分位
    comparableAge?: string;       // 相当年龄

    // 分项能力
    skills: SkillScore[];

    // 薄弱点
    weakPoints: WeakPoint[];

    // 强项
    strongPoints: StrongPoint[];

    // 对比分析
    comparison: ComparisonData;

    // 推荐行动
    recommendations: Recommendation[];

    // 报告元数据
    generatedAt: Date;
    validUntil: Date;
    shareUrl?: string;
}

```

```

// 分项能力得分
export interface SkillScore {
  dimension: AbilityDimension;
  score: number; // 0-100
  cefrLevel: CEFRLevel;
  description: string;
  strength: 1 | 2 | 3; // 1=弱 2=中 3=强
}

// 薄弱点
export interface WeakPoint {
  knowledgePoint: string;
  topic: string;
  correctRate: number;
  recommendation: string;
}

// 强项
export interface StrongPoint {
  knowledgePoint: string;
  topic: string;
  correctRate: number;
  description: string;
}

// 对比数据
export interface ComparisonData {
  vsStudents: {
    percentile: number;
    description: string;
  };
  vsTarget?: {
    targetLevel: CEFRLevel;
    gap: number;
    estimatedTime: string;
  };
}

// 推荐行动
export interface Recommendation {
  action: string;
  type: 'consult' | 'practice' | 'course' | 'material';
  description: string;
  priority: number;
  urgency: 'high' | 'medium' | 'low';
}

```


// 报告模块主类

```
export class ReportModule {  
  private report: DiagnosticReport | null = null;  
  
  // 公开API  
  public async generateReport(session: TestSession): Promise<DiagnosticReport>;  
  public getReport(): DiagnosticReport | null;  
  public async shareReport(platform: 'wechat' | 'moments' | 'xhs'): Promise<string>;  
  public async downloadReport(): Promise<string>; // 返回PDF URL  
  public renderReportPage(report: DiagnosticReport): React.ReactNode;  
}
```

// 报告生成算法

```
export class ReportGenerator {  
  private diagnosticModel: IRTModel;  
  private questionKnowledgeMap: Map<string, string[]>;  
  
  public async generate(session: TestSession): Promise<DiagnosticReport> {  
    // 1. 计算IRT能力参数  
    const theta = this.calculateAbilityTheta(session.answers);  
  
    // 2. 映射CEFR等级  
    const cefr = this.mapThetaToCEFR(theta);  
  
    // 3. 计算分项能力  
    const skills = this.calculateSkillScores(session);  
  
    // 4. 定位薄弱点  
    const weakPoints = this.identifyWeakPoints(session);  
  
    // 5. 识别强项  
    const strongPoints = this.identifyStrongPoints(session);  
  
    // 6. 生成推荐  
    const recommendations = this.generateRecommendations(cefr,  
weakPoints);  
  
    // 7. 组装报告  
    return this.assembleReport(session, theta, cefr, skills, weakPoints,  
strongPoints, recommendations);  
  }  
  
  private calculateAbilityTheta(answers: Map<string, AnswerRecord>): number {  
    // IRT能力估计 (使用EM算法或贝叶斯估计)  
    //  $\theta_{new} = \theta_{old} + \frac{\sum (a_i * (P_j - Y_{ij}))}{\sum (a_i^2 * P_i * Q_i)}$   
  }
```

```

typescript // 具体实现见IRTModel类
}

private mapThetaToCEFR(theta: number): CEFRLevel {
    // IRT theta -> CEFR映射表
    const mapping = [
        { theta: -2.0, cefr: CEFRLevel.A1 },
        { theta: -1.0, cefr: CEFRLevel.A2 },
        { theta: 0.0, cefr: CEFRLevel.B1 },
        { theta: 1.0, cefr: CEFRLevel.B2 },
        { theta: 2.0, cefr: CEFRLevel.C1 },
    ];

    for (const m of mapping) {
        if (theta < m.theta + 0.5) {
            return m.cefr;
        }
    }
    return CEFRLevel.C2;
}
}

```

8.3.3 转化模块 (ConversionModule)

```

// ===== 文件: src/modules/conversion/ConversionModule.ts
// =====

// 转化漏斗阶段
export enum FunnelStage {
    TEST_STARTED = 'test_started',
    TEST_COMPLETED = 'test_completed',
    REPORT_VIEWED = 'report_viewed',
    REPORT_SHARED = 'report_shared',
    CONSULT_BOOKED = 'consult_booked',
    CONSULT_COMPLETED = 'consult_completed',
    PRACTICE_STARTED = 'practice_started',
    PRACTICE_COMPLETED = 'practice_completed',
    PAID = 'paid'
}

// 意向度等级
export enum IntentionLevel {
    HIGH = 'high',

```

```

    MEDIUM = 'medium',
    LOW = 'low'
}

// 咨询预约
export interface ConsultBooking {
    bookingId: string;
    reportId?: string;
    userId?: string;

    // 联系人信息
    parentName: string;
    parentPhone: string;
    childName: string;
    childAge?: number;
    childGrade?: string;

    // 预约信息
    preferredTime?: string;
    intentionLevel: IntentionLevel;
    assignedSalesId?: string;

    // 状态
    status: 'pending' | 'confirmed' | 'completed' | 'cancelled';
    notes?: string;
}

// 转化模块主类
export class ConversionModule {
    private funnelTracker: FunnelTracker;
    private intentionScorer: IntentionScorer;
    private guidanceEngine: GuidanceEngine;

    // 公开API
    public trackStage(stage: FunnelStage, data?: object): Promise<void>;
    public calculateIntention(userId: string): Promise<IntentionLevel>;
    public bookConsult(booking: ConsultBooking): Promise<ConsultBooking>;
    public getGuidance(report: DiagnosticReport, userContext: object): GuidanceConfig;
    public showGuidancePopup(report: DiagnosticReport, userContext: object): void;
}

// 漏斗追踪器
export class FunnelTracker {
    public async track(userId: string, sessionId: string, stage: FunnelStage, data?:
object): Promise<void>;
    public async getFunnelProgress(userId: string): Promise<FunnelProgress>;

```

```

}
typecript

// 引导引擎
export interface GuidanceConfig {
  primaryAction: ActionConfig;
  secondaryActions: ActionConfig[];
  popup?: PopupConfig;
  banners?: BannerConfig[];
}

export interface ActionConfig {
  type: 'consult' | 'practice' | 'material' | 'share';
  label: string;
  description: string;
  icon?: string;
  deepLink?: string;
}

export interface PopupConfig {
  title: string;
  content: string;
  actions: ActionConfig[];
  triggerCondition: string;
  showOnce: boolean;
}

```

8.4 API接口设计

8.4.1 测试相关接口

```

# ===== 文件: api_spec/test_api.yaml =====

openapi: 3.0.0
info:
  title: 英语诊断测试API
  version: 1.0.0
  basePath: /api/v1

paths:
  # 创建测试会话
  /test/sessions:
    post:
      summary: 创建测试会话

```

```
tags:
  - Test
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required:
          - test_id
          - level
          - test_type
        properties:
          test_id:
            type: string
            example: KET_EXP_001
            description: 测试模板ID
          level:
            type: string
            enum: [KET, PET, FCE]
            description: 测试级别
          test_type:
            type: string
            enum: [experience, placement, full]
            description: 测试类型
          source_channel:
            type: string
            example: wechat
            description: 来源渠道
          ref_id:
            type: string
            description: 推荐人/分享者ID
          device_id:
            type: string
            description: 设备ID
responses:
  '201':
    description: 会话创建成功
    content:
      application/json:
        schema:
          type: object
          properties:
            session_id:
              type: string
```

```
        questions:
          type: array
          items:
            $ref: '#/components/schemas/Question'
        time_limit:
          type: integer
        start_url:
          type: string
```

获取下一题

/test/sessions/{session_id}/next:

```
get:
  summary: 获取下一题
  tags:
    - Test
  parameters:
    - name: session_id
      in: path
      required: true
      schema:
        type: string
    - name: current_theta
      in: query
      schema:
        type: number
      description: 当前能力估计(用于自适应选题)
  responses:
    '200':
      description: 成功
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Question'
    '204':
      description: 测试完成，无下一题
```

提交答案

/test/sessions/{session_id}/answers:

```
post:
  summary: 提交答案
  tags:
    - Test
  requestBody:
    required: true
    content:
      application/json:
```

```
    schema:
      type: object
      required:
        - question_id
        - answer
        - time_spent_ms
      properties:
        question_id:
          type: string
        answer:
          type: string
        time_spent_ms:
          type: integer
        attempt_count:
          type: integer
          default: 1
        change_count:
          type: integer
          default: 0
  responses:
    '200':
      description: 成功
      content:
        application/json:
          schema:
            type: object
            properties:
              is_correct:
                type: boolean
              correct_answer:
                type: string
              explanation:
                type: string
```

提交测试

/test/sessions/{session_id}/submit:

```
  post:
    summary: 提交测试
    tags:
      - Test
    responses:
      '200':
        description: 提交成功，开始生成报告
        content:
          application/json:
```

```

yaml
    schema:
      type: object
      properties:
        status:
          type: string
          example: processing
        report_id:
          type: string
        estimated_time:
          type: integer
          description: 预计生成时间(秒)

components:
  schemas:
    Question:
      type: object
      properties:
        id:
          type: string
        type:
          type: string
          enum: [choice, listening, cloze, reading]
        content:
          type: string
        options:
          type: array
          items:
            type: object
            properties:
              key:
                type: string
              text:
                type: string
        media_url:
          type: string
        time_limit:
          type: integer
        difficulty:
          type: number

```

8.4.2 报告相关接口


```
# ===== 文件: api_spec/report_api.yaml =====
```

```
paths:
```

```
  # 获取报告
```

```
  /reports/{report_id}:
```

```
    get:
```

```
      summary: 获取诊断报告
```

```
      tags:
```

```
        - Report
```

```
      parameters:
```

```
        - name: report_id
```

```
          in: path
```

```
          required: true
```

```
          schema:
```

```
            type: string
```

```
      responses:
```

```
        '200':
```

```
          description: 成功
```

```
          content:
```

```
            application/json:
```

```
              schema:
```

```
                $ref: '#/components/schemas/DiagnosticReport'
```

```
  # 分享报告
```

```
  /reports/{report_id}/share:
```

```
    post:
```

```
      summary: 分享报告
```

```
      tags:
```

```
        - Report
```

```
      requestBody:
```

```
        content:
```

```
          application/json:
```

```
            schema:
```

```
              type: object
```

```
              required:
```

```
                - platform
```

```
              properties:
```

```
                platform:
```

```
                  type: string
```

```
                  enum: [wechat, moments, xhs, douyin, copy_link]
```

```
      responses:
```

```
        '200':
```

```
          description: 成功
```

```
          content:
```

```
yaml      application/json:
          schema:
            type: object
            properties:
              share_link:
                type: string
              share_image:
                type: string
              share_title:
                type: string
              share_desc:
                type: string

components:
  schemas:
    DiagnosticReport:
      type: object
      properties:
        report_id:
          type: string
        ability_theta:
          type: number
        ceفر_level:
          type: string
        percentile:
          type: integer
        skills:
          type: array
          items:
            type: object
            properties:
              dimension:
                type: string
              score:
                type: number
              level:
                type: string
        weak_points:
          type: array
          items:
            $ref: '#/components/schemas/WeakPoint'
        recommendations:
          type: array
          items:
            $ref: '#/components/schemas/Recommendation'
```

8.5 数据库表结构(完整SQL)

```
-- ===== 文件: database/schema.sql =====

-- 题目表
CREATE TABLE questions (
  id          VARCHAR(32) PRIMARY KEY,
  level       VARCHAR(10) NOT NULL,   -- KET/PET/FCE
  type        VARCHAR(20) NOT NULL,   -- choice/listening/cloze/reading
  content     TEXT NOT NULL,
  options     JSONB,                  -- [{"key": "A", "text": "..."}]
  correct_answer VARCHAR(20) NOT NULL,
  explanation TEXT,

  -- IRT参数
  difficulty  DECIMAL(5,3) NOT NULL,  -- b参数
  discrimination DECIMAL(5,3) DEFAULT 1.0, -- a参数
  guessing    DECIMAL(5,3) DEFAULT 0.25, -- c参数

  -- 元数据
  knowledge_points TEXT[],             -- ["grammar_verb_tense",
  "vocab_irregular"]
  media_url      VARCHAR(256),
  time_limit_sec INTEGER DEFAULT 60,

  -- 状态
  status        VARCHAR(20) DEFAULT 'active',
  usage_count   INTEGER DEFAULT 0,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_level_type (level, type),
  INDEX idx_difficulty (difficulty),
  INDEX idx_knowledge (knowledge_points)
);

-- 答题日志表 (L1数据采集)
CREATE TABLE answer_logs (
  id          BIGSERIAL PRIMARY KEY,
  session_id  VARCHAR(64) NOT NULL,
  user_id     VARCHAR(64),
  openid      VARCHAR(128),
  test_id     VARCHAR(32) NOT NULL,
  question_id VARCHAR(32) NOT NULL,
```

```

question_seq    SMALLINT NOT NULL,

-- 答题数据
chosen_option   VARCHAR(10),
correct_option  VARCHAR(10),
is_correct      BOOLEAN,
time_spent_ms   INTEGER NOT NULL,
attempt_count   SMALLINT DEFAULT 1,
first_select_ms INTEGER,
change_count    SMALLINT DEFAULT 0,

-- 上下文
source_channel  VARCHAR(32),
ref_id          VARCHAR(64),
device_id       VARCHAR(128),

created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_session (session_id),
INDEX idx_user (user_id),
INDEX idx_question (question_id),
INDEX idx_created (created_at)
);

```

-- 测试会话表

```

CREATE TABLE test_sessions (
    id                BIGSERIAL PRIMARY KEY,
    session_id        VARCHAR(64) UNIQUE NOT NULL,
    user_id            VARCHAR(64),
    openid             VARCHAR(128),
    test_id            VARCHAR(32) NOT NULL,
    test_type          VARCHAR(20) NOT NULL,
    level              VARCHAR(10) NOT NULL,

-- 用户信息
    source_channel     VARCHAR(32),
    ref_id             VARCHAR(64),
    device_id          VARCHAR(128),
    device_type        VARCHAR(50),
    os_version          VARCHAR(20),
    app_version         VARCHAR(20),
    ip_address          VARCHAR(45),
    region              VARCHAR(50),

-- 测试状态
    status              VARCHAR(20) DEFAULT 'in_progress',

```

```

total_questions SMALLINT,
answered_count  SMALLINT DEFAULT 0,
correct_count   SMALLINT DEFAULT 0,
total_time_ms   INTEGER,

-- 快速定级结果
placement_theta DECIMAL(5,3),
recommended_level VARCHAR(10),

started_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
completed_at    TIMESTAMP,

INDEX idx_user (user_id),
INDEX idx_openid (openid),
INDEX idx_status (status),
INDEX idx_created (created_at)
);

-- 诊断报告表 (L4输出)
CREATE TABLE diagnostic_reports (
  id          BIGSERIAL PRIMARY KEY,
  report_id   VARCHAR(64) UNIQUE NOT NULL,
  session_id  VARCHAR(64) NOT NULL,
  user_id     VARCHAR(64),
  test_id     VARCHAR(32) NOT NULL,
  level       VARCHAR(10) NOT NULL,

  -- 能力画像
  ability_theta DECIMAL(5,3) NOT NULL,
  cefr_level    VARCHAR(5) NOT NULL,
  percentile    SMALLINT,
  comparable_age VARCHAR(20),

  -- 分项能力 (JSONB)
  skills        JSONB,

  -- 薄弱点强项
  weak_points   JSONB,
  strong_points JSONB,

  -- 完整报告数据
  report_data   JSONB,
  pdf_url       VARCHAR(256),

  -- 分享数据
  share_count   SMALLINT DEFAULT 0,

```

```

share_url          VARCHAR(256),

-- 转化状态
conversion_status  VARCHAR(20) DEFAULT 'report_viewed',

created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
expires_at         TIMESTAMP,

INDEX idx_user (user_id),
INDEX idx_session (session_id),
INDEX idx_cefr (cefr_level),
INDEX idx_conversion (conversion_status)
);

-- 咨询预约表
CREATE TABLE consult_bookings (
  id                BIGSERIAL PRIMARY KEY,
  booking_id        VARCHAR(64) UNIQUE NOT NULL,
  report_id         VARCHAR(64),
  user_id           VARCHAR(64),

  -- 联系信息
  parent_name       VARCHAR(50) NOT NULL,
  parent_phone      VARCHAR(20) NOT NULL,
  child_name        VARCHAR(50),
  child_age         SMALLINT,
  child_grade       VARCHAR(20),

  -- 意向
  intention_level   VARCHAR(20) DEFAULT 'medium',
  preferred_time    VARCHAR(100),
  notes            TEXT,

  -- 分配
  assigned_sales_id VARCHAR(64),

  -- 状态
  status            VARCHAR(20) DEFAULT 'pending',
  follow_up_count   SMALLINT DEFAULT 0,

  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  confirmed_at      TIMESTAMP,
  completed_at      TIMESTAMP,

  INDEX idx_phone (parent_phone),
  INDEX idx_status (status)

```

```
);
```

```
-- 转化漏斗表
```

```
CREATE TABLE conversion_funnel (  
  id BIGSERIAL PRIMARY KEY,  
  session_id VARCHAR(64) NOT NULL,  
  user_id VARCHAR(64),  
  report_id VARCHAR(64),  
  booking_id VARCHAR(64),  
  
  stage VARCHAR(30) NOT NULL,  
  stage_status VARCHAR(20) DEFAULT 'entered',  
  entered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  exited_at TIMESTAMP,  
  exit_reason VARCHAR(50),  
  
  action_type VARCHAR(30),  
  action_detail JSONB,  
  
  INDEX idx_session (session_id),  
  INDEX idx_user (user_id),  
  INDEX idx_stage (stage)  
);
```

```
-- 学员能力画像表 (L4长期存储)
```

```
CREATE TABLE ability_profiles (  
  id BIGSERIAL PRIMARY KEY,  
  user_id VARCHAR(64) UNIQUE NOT NULL,  
  
  -- 最新能力  
  current_theta DECIMAL(5,3),  
  current_cefr VARCHAR(5),  
  current_percentile SMALLINT,  
  last_test_date DATE,  
  last_report_id VARCHAR(64),  
  
  -- 能力历史  
  ability_history JSONB, -- [{"date": "2025-01-01", "theta": 0.5, "cefr": "A2"}, ...]  
  
  -- 能力趋势  
  trend_direction VARCHAR(10), -- improving/stable/declining  
  improvement_rate DECIMAL(5,3),  
  
  -- 薄弱点追踪  
  persistent_weak_points JSONB,
```

```

sql  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
     updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

     INDEX idx_cefr (current_cefr),
     INDEX idx_trend (trend_direction)
);

-- 用户表
CREATE TABLE users (
    id                BIGSERIAL PRIMARY KEY,
    user_id           VARCHAR(64) UNIQUE NOT NULL,
    openid            VARCHAR(128),
    unionid           VARCHAR(128),

    -- 基本信息
    nickname          VARCHAR(50),
    avatar_url        VARCHAR(256),

    -- 家长信息
    parent_name       VARCHAR(50),
    parent_phone      VARCHAR(20),

    -- 孩子信息
    child_name        VARCHAR(50),
    child_age         SMALLINT,
    child_grade       VARCHAR(20),

    -- 来源
    first_source       VARCHAR(32),
    first_ref_id      VARCHAR(64),

    -- 状态
    status            VARCHAR(20) DEFAULT 'active',
    is_paid           BOOLEAN DEFAULT FALSE,
    is_student        BOOLEAN DEFAULT FALSE,

    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_active_at    TIMESTAMP,

    INDEX idx_openid (openid),
    INDEX idx_phone (parent_phone),
    INDEX idx_is_paid (is_paid)
);

```

8.6 Coze组件定义

8.6.1 Coze Bot 定义

```
# ===== 文件: coze/bots/diagnosis_guide_bot.yaml
=====
```

bot_name: 英语诊断引导Bot

bot_description: 引导用户完成英语能力诊断测试的对话机器人

触发条件

trigger_conditions:

- type: keyword
keywords: ["测试", "诊断", "水平", "KET", "PET", "FCE"]
- type: intent
intents: ["start_test", "check_level"]
- type: event
events: ["user_first_visit"]

对话流程

dialog_flow:

greeting:

message: |
您好！我是英语诊断助手 🙋

我可以帮您：

- 3分钟测出孩子的英语真实水平
- 精确定位薄弱知识点
- 推荐个性化学习方案

您想现在就测试一下吗？

buttons:

- label: 开始免费测试
action: start_test
- label: 了解测试内容
action: show_info

show_info:

message: |
 **英语能力诊断测试**

测试内容：

- KET/PET/FCE真题
- AI自适应出题
- 即时生成报告

****测试时长: ****

- 快速定级: 3题/2分钟
- 体验版: 8题/5分钟
- 完整版: 25题/30分钟

****立即开始? ****

buttons:

- label: 快速定级测试
action: start_placement
- label: 体验版测试
action: start_experience
- label: 选择级别
action: select_level

start_test:

action: |

生成个性化测试链接

更新漏斗状态: test_starting

response: |

好的, 马上为您准备测试! 🎯

测试链接: [打开测试]

完成后会自动生成诊断报告哦~

consult_intent:

trigger_keywords: ["咨询", "预约", "分析", "了解课程"]

message: |

好的, 我来帮您预约专业咨询! 📝

请留下您的联系方式:

- 孩子姓名: ____
- 孩子年级: ____
- 联系电话: ____

顾问会在30分钟内联系您!

collect_fields:

- child_name
- child_grade
- parent_phone

on_complete: |

创建咨询预约

分配给销售顾问

更新漏斗状态: consult_booked

response: |

✅ 预约成功!

yaml

我们的学习规划师会在30分钟内
通过电话联系您: {{parent_phone}}

同时, 您可以先让孩子做一个
免费的英语能力测试~

变量定义

variables:

user_level: string # 用户CEFR等级
test_session_id: string
report_id: string
consult_booked: boolean

意图识别

intents:

- name: start_test
patterns:
 - "开始测试"
 - "测试一下"
 - "测测水平"
 - "做测试"
- name: check_report
patterns:
 - "查看报告"
 - "报告在哪"
 - "诊断结果"
- name: book_consult
patterns:
 - "预约咨询"
 - "联系顾问"
 - "人工服务"

8.6.2 Coze Workflow 定义

===== 文件: coze/workflows/report_generation.yaml
=====

workflow_name: 诊断报告生成
workflow_id: wf_report_generation
description: 从答题数据生成完整诊断报告

输入参数

input_params:

session_id:

type: string

required: true

user_id:

type: string

required: false

流程步骤

steps:

- id: step_1

name: 获取答题数据

type: database_query

config:

table: answer_logs

query: |

SELECT * FROM answer_logs

WHERE session_id = \$session_id

ORDER BY question_seq

output: answer_data

- id: step_2

name: 计算IRT能力参数

type: code

config:

language: python

code: |

from irt_model import IRTEstimator

answers = {{step_1.output}}

estimator = IRTEstimator()

theta = estimator.estimate(answers)

return {"theta": theta}

output: ability_theta

- id: step_3

name: 映射CEFR等级

type: code

config:

language: python

code: |

theta = {{step_2.theta}}

```

    ceفر_map = [
        (-2.0, "A1"),
        (-1.0, "A2"),
        (0.0, "B1"),
        (1.0, "B2"),
        (2.0, "C1"),
    ]

    for threshold, ceفر in ceفر_map:
        if theta < threshold + 0.5:
            return {"ceفر": ceفر}
    return {"ceفر": "C2"}
output: ceفر_level

- id: step_4
  name: 计算分项能力
  type: database_query
  config:
    table: questions
    query: |
SELECT q.id, q.knowledge_points, a.is_correct
FROM answer_logs a
JOIN questions q ON a.question_id = q.id
WHERE a.session_id = $session_id
  output: skill_data

- id: step_5
  name: 生成薄弱点分析
  type: code
  config:
    language: python
    code: |
# 按知识点分组计算正确率
skill_data = {{step_4.output}}

from collections import defaultdict
kp_stats = defaultdict(lambda: {"correct": 0, "total": 0})

for item in skill_data:
    for kp in item.knowledge_points:
        kp_stats[kp]["total"] += 1
        if item.is_correct:
            kp_stats[kp]["correct"] += 1

weak_points = [
    {"knowledge_point": k, "correct_rate": v["correct"]/v["total"]}

```

```

        for k, v in kp_stats.items()
        if v["total"] >= 2 and v["correct"]/v["total"] < 0.6
    ]

    return {"weak_points": sorted(weak_points, key=lambda x: x["correct_rate"])}
output: weak_points

- id: step_6
  name: 生成推荐行动
  type: code
  config:
    language: python
    code: |
cefr = {{step_3.cefr}}
weak_points = {{step_5.weak_points}}

    recommendations = []

    # 基于CEFR的推荐
    if cefr in ["A1", "A2"]:
        recommendations.append({
            "action": "预约KET备考课程",
            "type": "course",
            "priority": 1,
            "urgency": "high"
        })
    elif cefr in ["B1", "B2"]:
        recommendations.append({
            "action": "预约PET/FCE冲刺班",
            "type": "course",
            "priority": 1,
            "urgency": "high"
        })

    # 基于薄弱点的推荐
    for wp in weak_points[:2]:
        recommendations.append({
            "action": f"专项练习: {wp['knowledge_point']}",
            "type": "practice",
            "priority": 2,
            "urgency": "medium"
        })

    return {"recommendations": recommendations}
output: recommendations

```

```

- id: step_7
  name: 组装报告数据
  type: code
  config:
    language: python
    code: |
import uuid
from datetime import datetime, timedelta

    report = {
        "report_id": f"rpt_{uuid.uuid4().hex[:12]}",
        "session_id": "{{input.session_id}}",
        "user_id": "{{input.user_id}}",
        "ability_theta": {{step_2.theta}},
        "cefr_level": {{step_3.cefr}},
        "skills": {{step_4.output}},
        "weak_points": {{step_5.weak_points}},
        "recommendations": {{step_6.recommendations}},
        "generated_at": datetime.now().isoformat(),
        "valid_until": (datetime.now() + timedelta(days=7)).isoformat()
    }

    return report
  output: report_data

- id: step_8
  name: 保存报告
  type: database_insert
  config:
    table: diagnostic_reports
    data: "{{step_7.output}}"
  output: save_result

- id: step_9
  name: 更新漏斗状态
  type: database_update
  config:
    table: conversion_funnel
    query: |
UPDATE conversion_funnel
SET stage = 'report_generated',
    stage_status = 'completed'
WHERE session_id = $session_id
  output: funnel_updated

```

输出

```
output:
  report_id: "{{step_7.report_id}}"
  cejr_level: "{{step_3.cejr}}"
  report_url: "/pages/report/detail?report_id={{step_7.report_id}}"
```

8.6.3 Coze Plugin 定义

```
# ===== 文件: coze/plugins/question_api.yaml =====
```

```
plugin_name: 题库API插件
plugin_id: plugin_question_api
description: 提供题目获取、答案验证等题库相关功能

api_endpoints:
  - name: 获取题目
    path: /questions/{question_id}
    method: GET
    description: 根据ID获取题目详情
    parameters:
      - name: question_id
        in: path
        required: true
        type: string
    response:
      type: Question
      example:
        id: "q_001"
        type: "choice"
        content: "She ____ to school every morning."
        options:
          - key: "A"
            text: "walk"
          - key: "B"
            text: "walks"
          - key: "C"
            text: "walking"
          - key: "D"
            text: "walked"
        correct_answer: "B"
        difficulty: 0.45

  - name: 自适应选题
    path: /questions/adaptive
```


method: POST
description: 根据用户能力自动选择下一题
parameters:

- name: level
in: body
required: true
type: string
enum: [KET, PET, FCE]
- name: current_theta
in: body
required: true
type: number
- name: answered_ids
in: body
required: true
type: array
items:
type: string

response:
type: Question

- name: 验证答案
path: /answers/verify
method: POST
description: 验证用户答案并返回结果
parameters:

- name: question_id
in: body
required: true
type: string
- name: user_answer
in: body
required: true
type: string

response:
type: object
properties:
is_correct:
type: boolean
correct_answer:
type: string
explanation:
type: string
description: 答案解析

===== 文件: coze/plugins/iflytek_speech.yaml =====

yaml

plugin_name: 讯飞语音评测插件
plugin_id: plugin_iflytek_speech
description: 集成讯飞语音评测API，支持口语能力评估

api_endpoints:

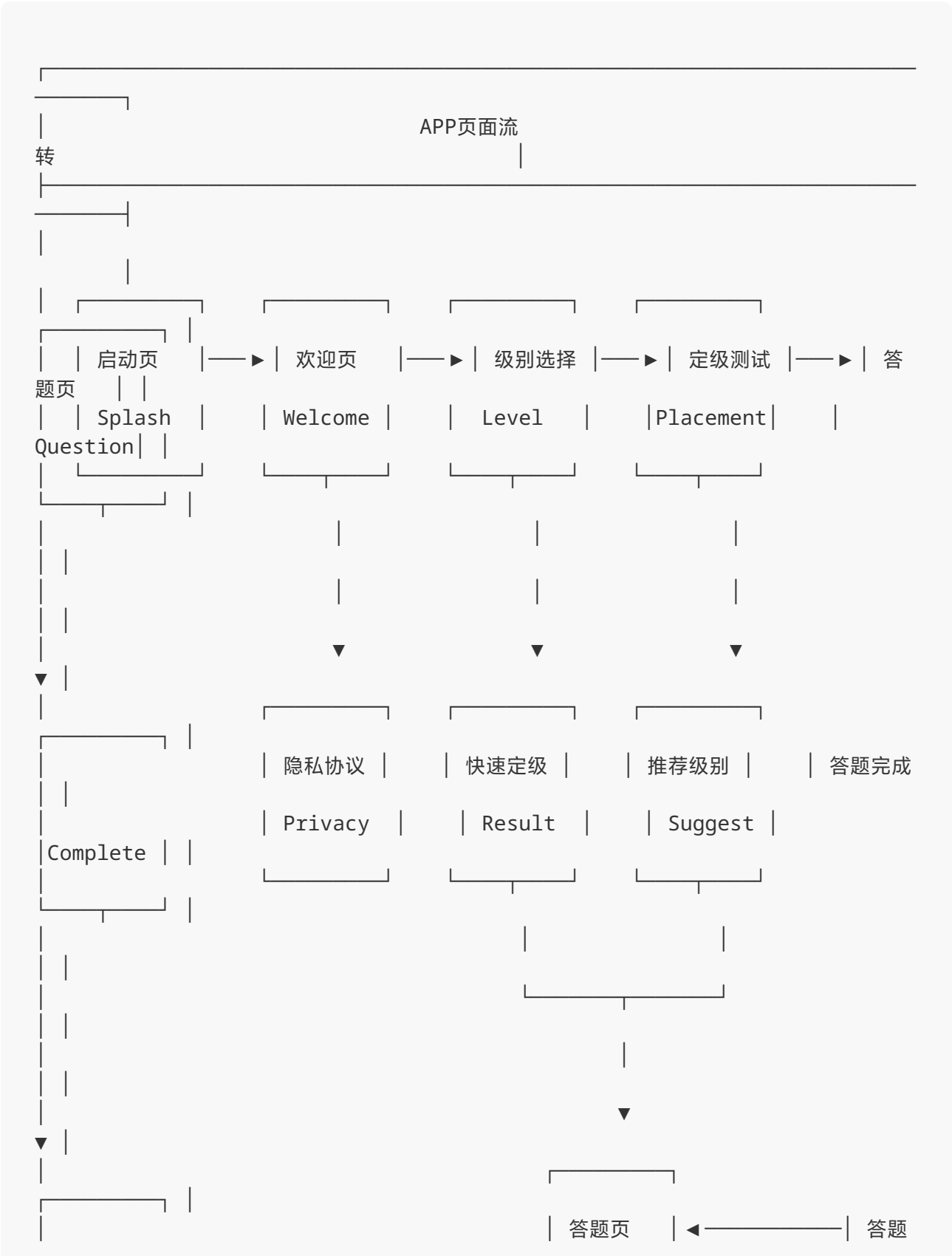
- name: 语音评测
path: /speech/evaluate
method: POST
description: 对用户口语进行AI评测
parameters:
 - name: audio_url
in: body
required: true
type: string
description: 音频文件URL
 - name: reference_text
in: body
required: true
type: string
description: 参考文本
 - name: scoring_model
in: body
required: false
type: string
default: "english_sentence"
description: 评测模型

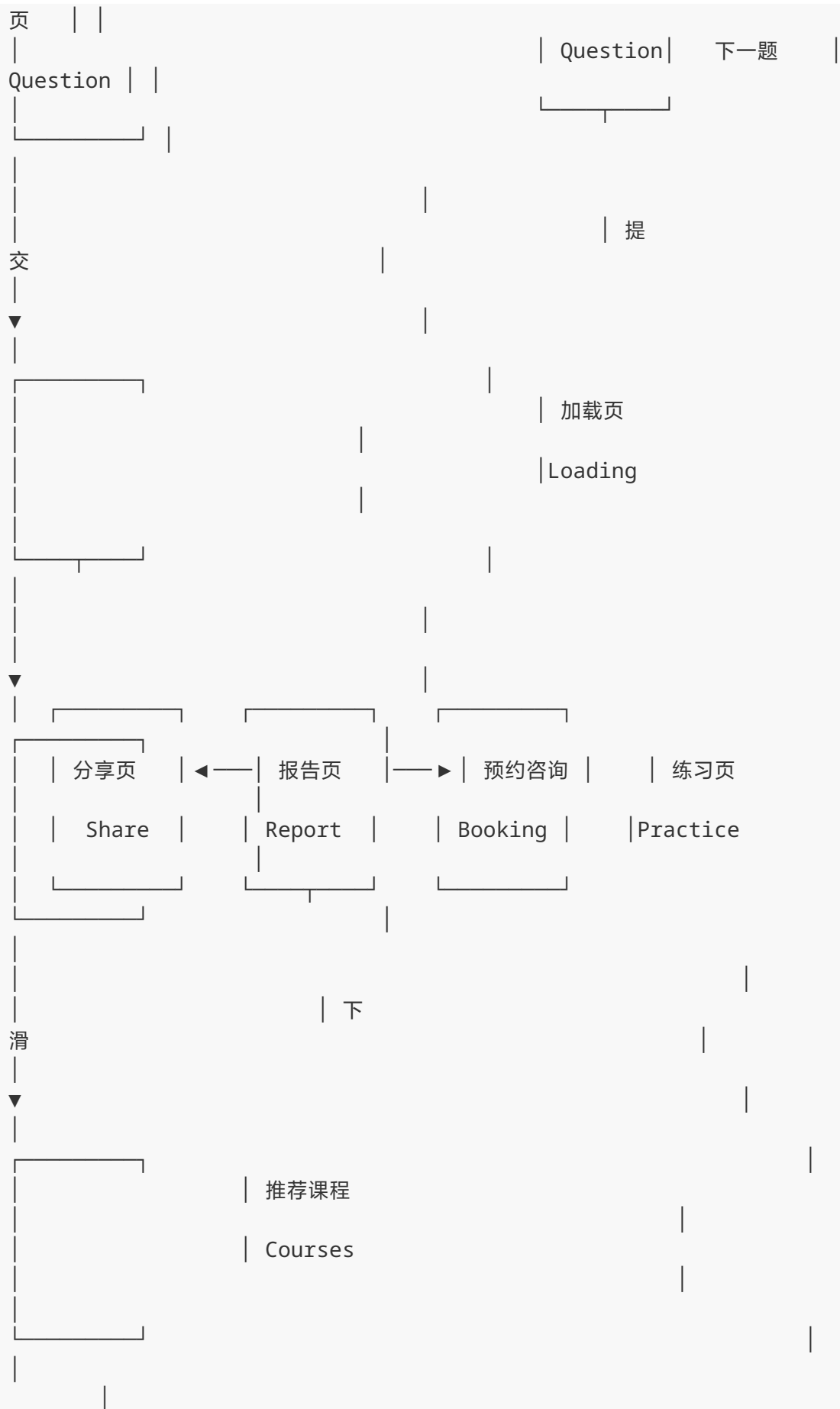
response:

type: object
properties:
total_score:
type: number
description: 总分(0-100)
pronunciation_score:
type: number
description: 发音分数
fluency_score:
type: number
description: 流利度分数
integrity_score:
type: number
description: 完整度分数
feedback:
type: string
description: 评测反馈

8.7 UI/UX规范

8.7.1 页面流转图





8.7.2 设计规范

```
/* ===== 文件: src/styles/design_tokens.css ===== */

/* 主题色 */
:root {
  /* 主色 */
  --primary-color: #4A90E2;          /* 品牌蓝 */
  --primary-dark: #357ABD;
  --primary-light: #6BA3E8;

  /* 辅助色 */
  --success-color: #52C41A;          /* 正确/成功 */
  --warning-color: #FAAD14;          /* 警告/中等 */
  --error-color: #FF4D4F;            /* 错误/薄弱 */

  /* 背景色 */
  --bg-color: #F7F8FA;
  --bg-white: #FFFFFF;
  --bg-card: #FFFFFF;
  --bg-disabled: #E8E8E8;

  /* 文字色 */
  --text-primary: #1A1A1A;
  --text-secondary: #666666;
  --text-placeholder: #999999;
  --text-disabled: #CCCCCC;

  /* 字体 */
  --font-family: -apple-system, BlinkMacSystemFont, 'PingFang SC', 'Hiragino
  Sans GB', 'Microsoft YaHei', sans-serif;
  --font-size-xs: 12px;
  --font-size-sm: 14px;
  --font-size-md: 16px;
  --font-size-lg: 18px;
  --font-size-xl: 20px;
  --font-size-xxl: 24px;
  --font-size-hero: 32px;

  /* 间距 */
}
```

```

--spacing-xs: 4px;
--spacing-sm: 8px;
--spacing-md: 16px;
--spacing-lg: 24px;
--spacing-xl: 32px;
--spacing-xxl: 48px;

/* 圆角 */
--radius-sm: 4px;
--radius-md: 8px;
--radius-lg: 12px;
--radius-xl: 16px;
--radius-full: 9999px;

/* 阴影 */
--shadow-sm: 0 2px 4px rgba(0, 0, 0, 0.08);
--shadow-md: 0 4px 12px rgba(0, 0, 0, 0.12);
--shadow-lg: 0 8px 24px rgba(0, 0, 0, 0.16);

/* 动画 */
--transition-fast: 150ms ease;
--transition-normal: 300ms ease;
--transition-slow: 500ms ease;
}

/* 组件样式 */
.btn-primary {
  background-color: var(--primary-color);
  color: white;
  border-radius: var(--radius-lg);
  padding: var(--spacing-md) var(--spacing-lg);
  font-size: var(--font-size-md);
  font-weight: 600;
  border: none;
  cursor: pointer;
  transition: all var(--transition-fast);
}

.btn-primary:hover {
  background-color: var(--primary-dark);
  transform: translateY(-1px);
  box-shadow: var(--shadow-md);
}

.btn-primary:active {
  transform: translateY(0);

```

```
css}

.card {
  background: var(--bg-card);
  border-radius: var(--radius-lg);
  padding: var(--spacing-lg);
  box-shadow: var(--shadow-sm);
}

.progress-bar {
  height: 8px;
  background: var(--bg-disabled);
  border-radius: var(--radius-full);
  overflow: hidden;
}

.progress-bar-fill {
  height: 100%;
  background: linear-gradient(90deg, var(--primary-color), var(--primary-light));
  border-radius: var(--radius-full);
  transition: width var(--transition-normal);
}
```

8.7.3 答题页交互规范

```
// ===== 文件: src/components/QuestionCard/QuestionCard.tsx
=====

import React, { useState, useEffect, useRef } from 'react';
import { View, Text, TouchableOpacity, Animated, StyleSheet } from 'react-native';

interface QuestionCardProps {
  question: Question;
  questionNumber: number;
  totalQuestions: number;
  onAnswer: (answer: string, timeSpentMs: number) => void;
  onTimeUp: () => void;
  timeLimit?: number; // 毫秒
}

export const QuestionCard: React.FC<QuestionCardProps> = ({
  question,
```

```

questionNumber,
totalQuestions,
onAnswer,
onTimeUp,
timeLimit
}) => {
  const [selectedAnswer, setSelectedAnswer] = useState<string |
null>(null);
  const [timeRemaining, setTimeRemaining] = useState(timeLimit || 0);
  const [firstSelectTime, setFirstSelectTime] = useState<number |
null>(null);
  const [changeCount, setChangeCount] = useState(0);
  const startTimeRef = useRef(Date.now());
  const animatedValue = useRef(new Animated.Value(1)).current;

  // 计时器
  useEffect(() => {
    if (!timeLimit) return;

    const timer = setInterval(() => {
      setTimeRemaining(prev => {
        if (prev <= 1000) {
          clearInterval(timer);
          onTimeUp();
          return 0;
        }
        return prev - 1000;
      });
    }, 1000);

    return () => clearInterval(timer);
  }, [timeLimit, onTimeUp]);

  // 选择答案
  const handleSelect = (optionKey: string) => {
    const now = Date.now();
    const timeSpent = now - startTimeRef.current;

    // 记录首次选择时间
    if (!firstSelectTime) {
      setFirstSelectTime(timeSpent);
    } else if (selectedAnswer && selectedAnswer !== optionKey) {
      // 答案修改
      setChangeCount(prev => prev + 1);
    }
  }

```



```

// 选中动画
Animated.sequence([
  Animated.timing(animatedValue, {
    toValue: 0.95,
    duration: 100,
    useNativeDriver: true
  }),
  Animated.timing(animatedValue, {
    toValue: 1,
    duration: 100,
    useNativeDriver: true
  })
]).start();

setSelectedAnswer(optionKey);

// 自动提交(可选)
// onAnswer(optionKey, timeSpent);
};

// 确认答案
const handleConfirm = () => {
  if (!selectedAnswer) return;
  const timeSpent = Date.now() - startTimeRef.current;
  onAnswer(selectedAnswer, timeSpent);
};

// 格式化时间
const formatTime = (ms: number) => {
  const minutes = Math.floor(ms / 60000);
  const seconds = Math.floor((ms % 60000) / 1000);
  return `${minutes}:${seconds.toString().padStart(2, '0')}`;
};

return (
  <View style={styles.container}>
    { /* 顶部进度 */ }
    <View style={styles.progressSection}>
      <Text style={styles.progressText}>
        {questionNumber} / {totalQuestions}
      </Text>
      {timeLimit > 0 && (
        <View style={styles.timerContainer}>
          <Text style={styles.timerText}>
            ⌚ {formatTime(timeRemaining)}
          </Text>
        </View>
      )}
    </View>
  </View>
);

```

```

    </View>
  })
</View>

{/* 进度条 */}
<View style={styles.progressBar}>
  <View
    style={[
      styles.progressFill,
      { width: `${(questionNumber / totalQuestions) * 100}%` }
    ]}
  />
</View>

{/* 题目内容 */}
<Animated.View style={[styles.questionCard, { transform: [{ scale: animatedValue }] }]}>
  <Text style={styles.questionText}>{question.content}</Text>

  {/* 听力播放按钮 */}
  {question.type === 'listening' && (
    <TouchableOpacity style={styles.audioButton}>
      <Text>🔊 播放音频</Text>
    </TouchableOpacity>
  )}
</Animated.View>

{/* 选项列表 */}
<View style={styles.optionsContainer}>
  {question.options?.map((option, index) => (
    <TouchableOpacity
      key={option.key}
      style={[
        styles.optionButton,
        selectedAnswer === option.key && styles.optionSelected
      ]}
      onPress={() => handleSelect(option.key)}
    >
      <View style={[
        styles.optionKey,
        selectedAnswer === option.key && styles.optionKeySelected
      ]}>
        <Text style={styles.optionKeyText}>
          {option.key}
        </Text>
      </View>
    </View>
  )}

```

```

        <Text style={[
          styles.optionText,
          selectedAnswer === option.key && styles.optionTextSelected
        ]}>
          {option.text}
        </Text>
      </TouchableOpacity>
    )}
  </View>

  {/* 确认按钮 */}
  <TouchableOpacity
    style={[
      styles.confirmButton,
      !selectedAnswer && styles.confirmButtonDisabled
    ]}
    onPress={handleConfirm}
    disabled={!selectedAnswer}
  >
    <Text style={styles.confirmButtonText}>确认答案</Text>
  </TouchableOpacity>
</View>
);
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: '#F7F8FA'
  },
  progressSection: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 8
  },
  progressText: {
    fontSize: 14,
    color: '#666'
  },
  timerContainer: {
    backgroundColor: '#FFF',
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 12
  }
});

```

```

},
timerText: {
  fontSize: 14,
  color: '#4A90E2',
  fontWeight: '600'
},
progressBar: {
  height: 4,
  backgroundColor: '#E8E8E8',
  borderRadius: 2,
  marginBottom: 24
},
progressFill: {
  height: '100%',
  backgroundColor: '#4A90E2',
  borderRadius: 2
},
questionCard: {
  backgroundColor: 'FFF',
  borderRadius: 16,
  padding: 24,
  marginBottom: 24,
  shadowColor: '000',
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.08,
  shadowRadius: 8,
  elevation: 3
},
questionText: {
  fontSize: 18,
  color: '#1A1A1A',
  lineHeight: 28,
  fontWeight: '500'
},
audioButton: {
  marginTop: 16,
  paddingVertical: 12,
  paddingHorizontal: 20,
  backgroundColor: 'F0F7FF',
  borderRadius: 8,
  alignSelf: 'flex-start'
},
optionsContainer: {
  gap: 12
},
optionButton: {

```

```
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#FFF',
    borderRadius: 12,
    padding: 16,
    borderWidth: 2,
    borderColor: '#E8E8E8'
  },
  optionSelected: {
    borderColor: '#4A90E2',
    backgroundColor: '#F0F7FF'
  },
  },
  optionKey: {
    width: 32,
    height: 32,
    borderRadius: 16,
    backgroundColor: '#E8E8E8',
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 12
  },
  },
  optionKeySelected: {
    backgroundColor: '#4A90E2'
  },
  },
  optionKeyText: {
    fontSize: 14,
    fontWeight: '600',
    color: '#666'
  },
  },
  optionText: {
    fontSize: 16,
    color: '#1A1A1A',
    flex: 1
  },
  },
  optionTextSelected: {
    color: '#4A90E2',
    fontWeight: '500'
  },
  },
  confirmButton: {
    marginTop: 24,
    backgroundColor: '#4A90E2',
    borderRadius: 12,
    paddingVertical: 16,
    alignItems: 'center'
  },
  },
  confirmButtonDisabled: {
```

```
typescript    backgroundColor: '#E8E8E8'
    },
    confirmButtonText: {
      fontSize: 16,
      fontWeight: '600',
      color: '#FFF'
    }
  });
```

9. 数据安全与隐私

9.1 隐私合规框架

```
// ===== 文件: src/utils/privacy.ts =====

/**
 * 未成年人数据保护规范
 * 依据：《个人信息保护法》、《未成年人保护法》、《儿童个人信息网络保护规定》
 */

// 隐私协议配置
export const PRIVACY_CONFIG = {
  // 未成年人年龄阈值
  MINOR_AGE: 14, // 14岁以下为儿童

  // 家长知情要求
  REQUIRES_PARENTAL_CONSENT: true,

  // 数据保留期限
  DATA_RETENTION: {
    // 未成年人：账户注销后6个月内删除
    minor: 6 * 30 * 24 * 60 * 60 * 1000,
    // 成年人：账户注销后30天内删除
    adult: 30 * 24 * 60 * 60 * 1000
  },

  // 敏感数据列表
  SENSITIVE_DATA: [
    'child_name',
    'child_age',
    'child_grade',
```

```

    'parent_phone',
    'speech_data', // 语音数据
    'test_results' // 测试结果
  ],

  // 数据用途限制
  DATA_USAGE_RESTRICTIONS: [
    '仅用于英语能力诊断服务',
    '未经家长同意不得用于营销',
    '不得向第三方共享未成年人数据'
  ]
};

// 年龄验证组件
export const AgeVerification: React.FC = {
  // 检测用户是否为未成年人
  checkMinorStatus(userData: UserData): boolean {
    const age = userData.childAge || 0;
    return age < PRIVACY_CONFIG.MINOR_AGE;
  },

  // 检查家长授权状态
  async checkParentalConsent(userId: string): Promise<boolean> {
    const consent = await db.query(
      'SELECT * FROM parental_consents WHERE user_id = $1 AND status = $2',
      [userId, 'approved']
    );
    return consent.length > 0;
  }
};

// 家长授权弹窗
export const ParentalConsentModal: React.FC = {
  content: `
    <div class="consent-modal">
      <h2>家长知情同意书</h2>
      <div class="consent-content">
        <h3>尊敬的家長: </h3>
        <p>感謝您信任我們的英語能力診斷服務。</p>
        <p>根據《個人信息保護法》規定，我們需要您: </p>
        <ol>
          <li>確認了解測試內容和目的</li>
          <li>同意我們收集和處理您孩子的學習數據</li>
          <li>授權我們使用數據生成診斷報告</li>
        </ol>
        <p><strong>家長確認碼: </strong> _____</p>
      </div>
    </div>
  `;
};

```

```
typescript <p> ( 由系統發送至家長手機 ) </p>
    </div>
    <div class="consent-checkbox">
        <input type="checkbox" id="consent">
        <label for="consent">
            我已閱讀並同意《隱私政策》和《家長知情同意書》
        </label>
    </div>
</div>
`,

    requiredFields: [
        'parent_phone',
        'parent_name',
        'verification_code',
        'consent_checkbox'
    ]
};

// 数据加密配置
export const ENCRYPTION_CONFIG = {
    // 传输加密
    transport: 'TLS 1.3',

    // 存储加密
    storage: {
        algorithm: 'AES-256-GCM',
        key_rotation: '90days'
    },

    // 语音数据特殊处理
    speech_data: {
        encryption: true,
        retention: '30days',
        anonymization: true
    }
};
```

9.2 数据删除机制

```
-- ===== 文件: database/procedures/data_deletion.sql
=====
```



```

-- 用户注销时的数据删除存储过程
CREATE OR REPLACE FUNCTION fn_delete_user_data(p_user_id VARCHAR)
RETURNS VOID AS $$
DECLARE
    v_is_minor BOOLEAN;
    v_retention_days INTEGER;
BEGIN
    -- 检查是否为未成年人
    SELECT child_age < 14 INTO v_is_minor FROM users WHERE user_id =
p_user_id;

    -- 确定保留期限
    IF v_is_minor THEN
        v_retention_days := 180; -- 6个月
    ELSE
        v_retention_days := 30; -- 30天
    END IF;

    -- 1. 匿名化用户个人信息
    UPDATE users SET
        nickname = '已注销用户',
        parent_name = NULL,
        parent_phone = CONCAT('deleted_', user_id),
        child_name = NULL,
        avatar_url = NULL,
        status = 'deleted',
        deleted_at = CURRENT_TIMESTAMP
    WHERE user_id = p_user_id;

    -- 2. 保留答题记录用于统计分析（匿名化）
    UPDATE answer_logs SET
        user_id = NULL,
        device_id = CONCAT('anon_', SUBSTRING(device_id, 1, 8))
    WHERE user_id = p_user_id;

    -- 3. 保留报告用于统计分析（匿名化）
    UPDATE diagnostic_reports SET
        user_id = NULL
    WHERE user_id = p_user_id;

    -- 4. 删除咨询预约（敏感信息）
    DELETE FROM consult_bookings WHERE user_id = p_user_id;

    -- 5. 删除会话数据
    DELETE FROM test_sessions WHERE user_id = p_user_id;

```

```

sql  -- 6. 安排延迟删除 ( 敏感文件 )
INSERT INTO scheduled_deletions (user_id, deletion_type, execute_at)
VALUES
    (p_user_id, 'files', CURRENT_TIMESTAMP + INTERVAL '1 day'),
    (p_user_id, 'voice_recordings', CURRENT_TIMESTAMP + INTERVAL '1 day');

-- 7. 记录操作日志
INSERT INTO audit_logs (action, user_id, details, executed_at)
VALUES ('user_data_deletion', p_user_id,
    json_build_object('is_minor', v_is_minor, 'retention_days',
v_retention_days),
    CURRENT_TIMESTAMP);

END;
$$ LANGUAGE plpgsql;

-- 敏感文件延迟删除表
CREATE TABLE scheduled_deletions (
    id BIGSERIAL PRIMARY KEY,
    user_id VARCHAR(64) NOT NULL,
    deletion_type VARCHAR(50) NOT NULL,
    file_paths JSONB,
    execute_at TIMESTAMP NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',
    executed_at TIMESTAMP,
    CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- 审计日志表
CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    action VARCHAR(100) NOT NULL,
    user_id VARCHAR(64),
    admin_id VARCHAR(64),
    details JSONB,
    ip_address VARCHAR(45),
    user_agent TEXT,
    executed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_action (action),
    INDEX idx_user (user_id),
    INDEX idx_executed (executed_at)
);

```

附录

A. 技术栈汇总

层级	技术选型	说明
前端框架	React Native 0.73+	跨平台移动开发
状态管理	Redux Toolkit	全局状态
路由	React Navigation 6	页面导航
HTTP客户端	Axios	API请求
数据分析	Amplitude + Mixpanel	用户行为分析
推送	极光推送	消息推送
登录	微信OpenSDK	微信授权
分享	微信ShareSDK	社交分享
语音评测	讯飞语音评测API	口语能力评估
后端	Coze Bot + Workflow	业务流程
数据库	PostgreSQL 15	主数据库
缓存	Redis 7	会话缓存
对象存储	OSS/S3	文件存储

B. 版本历史

版本	日期	修改内容
v1.0	2025-01-14	初始版本

C. 联系方式

技术支持: support@example.com

产品反馈: product@example.com

